Landkarten aus Geodaten mit Ti*k*Z zeichnen

Fortbildungsmaterial für Lehrkräfte

11. November 2026

Stumme Karten zum Beschriften, eine Lagekarte für ein Projekt in Sachsen, die Besatzungszonen von 1945, Kartenprojektionen im Vergleich oder ein Algorithmus, der Umrisse vereinfacht: Karten werden in Geografie, Geschichte, Gemeinschaftskunde, Mathematik und Informatik gebraucht. Eine Karte „aus dem Kopf“ mit Ti*k*Z-Koordinaten zu zeichnen, gelingt aber nie überzeugend. Diese Handreichung zeigt einen anderen Weg: Freie Geodaten werden mit einem Python-Skript in Ti*k*Z-Pfade umgerechnet. Danach lässt sich jedes Land, jeder Fluss, jede Autobahn und jede Höhenstufe einzeln gestalten.

Die Kapitel 1 bis 4 erklären die Grundlagen: Polygonzüge, Geodaten, Projektionen und Vereinfachung. Kapitel 6 zeigt, wie man mit dem Werkzeug `geo2tikz.py` eigene Karten für einen beliebigen Ausschnitt baut. Die Kapitel 7 bis 10 stellen einen fertigen Atlas mit zehn Beispielkarten vor, von Deutschland mit Höhenschichten bis zum Römischen Reich. Am Ende stehen Unterrichtsideen, ein Katalog freier Datenquellen und die wichtigsten Stolpersteine.

Inhaltsverzeichnis

1 Die Idee: Ein Land ist ein Polygonzug **4**

2 Woher bekommt man die Koordinaten? **4**

3 Vom Globus aufs Papier: die Projektion **5**

4 Vereinfachen: der Douglas-Peucker-Algorithmus **6**

5 Zwei Werkzeuge **7**

6 Eigene Karten mit `geo2tikz.py` 8

6.1 Daten herunterladen . . . 8

6.2 Der Aufruf . . . 9

6.3 Was in der erzeugten Datei steht . . . 9

6.4 Die erste Karte . . . 10

6.5 Mehrere Ebenen, Hervorhebungen und Städte . . . 11

6.6 Eine stumme Karte . . . 12

7 Der Beispiel-Atlas 13

7.1 Die Befehle der Datendateien . . . 14

7.2 Lagekarte: Sachsen in Deutschland . . . 15

7.3 Deutschland mit Bundesländern, Flüssen und Seen . . . 15

7.4 USA mit Bundesstaaten . . . 17

8 Mehr Ebenen für Deutschland: Gebirge, Höhen, Verkehr 19

8.1 Höhenschichten aus einem Höhenmodell . . . 20

8.2 Autobahnen und Bahnstrecken . . . 22

9 Europa und die ganze Welt 23

9.1 Europa: Färben ist ein Graphenproblem . . . 23

9.2 Weltkarten: Jede Projektion lügt, aber anders . . . 25

10 Historische Karten 27

10.1 Das Römische Reich um 200 n. Chr. . . . 27

10.2 Deutschland 1938 – 1945 – 1949 bis 1990 28

11 Und wenn ich nur eine Grafik habe? 30

12 Ideen für den Unterricht 32

13 Datenquellen-Katalog 33

14 Tipps und Stolpersteine 36

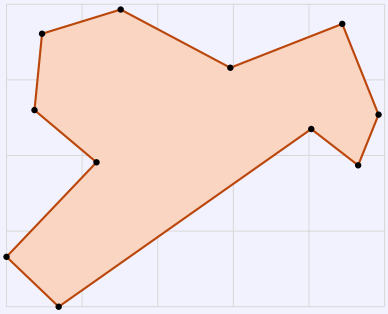
15 Zusammenfassung 37

1 Die Idee: Ein Land ist ein Polygonzug

Jede Grenze auf einer Landkarte ist mathematisch betrachtet ein **geschlossener Polygonzug**: eine Folge von Punkten $P_1, P_2, \ldots, P_n$, die nacheinander durch Strecken verbunden werden; der letzte Punkt wird wieder mit dem ersten verbunden. Genau das kann Ti*k*Z mit `--` und `cycle`. Eine sehr grobe Fassung von Sachsen mit nur elf Punkten:

```
\begin{tikzpicture}
  \draw[help lines, black!15] (0,0) grid (5,4);
  \draw[fill=akzentorange!25, draw=akzentorange!80!black, thick, line join=round]
    (4.03,2.35) -- (0.69,0.00) -- (0.00,0.66) -- (1.19,1.91) -- (0.37,2.60) -- (0.47,3.61)
    -- (1.51,3.93) -- (2.96,3.16) -- (4.44,3.74) -- (4.92,2.54) -- (4.65,1.87) -- cycle;
  \foreach \p in {(4.03,2.35),(0.69,0.00),(0.00,0.66),(1.19,1.91),(0.37,2.60),(0.47,3.61),
                  (1.51,3.93),(2.96,3.16),(4.44,3.74),(4.92,2.54),(4.65,1.87)}
    \fill \p circle (1.2pt);
  \node[font=\scriptsize, black!60] at (2.5,-0.35) {1 Kästchen $\hat{=}$ 1\,cm $\hat{=}$ ca.\
      43\,km};
\end{tikzpicture}
```

Ausgabe



1 Kästchen ≙ 1 cm ≙ ca. 43 km

Für eine *echte* Karte braucht man nicht 11, sondern einige hundert bis tausend Punkte pro Land, und die tippt niemand von Hand ab. Man holt sie aus frei verfügbaren **Geodaten** und lässt sie von einem Skript in Ti*k*Z-Koordinaten umrechnen.

Merke:

Eine Ti*k*Z-Karte besteht aus drei Zutaten: (1) Koordinaten in Längen- und Breitengrad aus einer Geodatenbank, (2) eine **Projektion**, die die Kugel auf das Papier abbildet, und (3) eine **Vereinfachung**, die überflüssige Punkte weglässt. Danach ist alles nur noch `\draw ... -- ... -- cycle;`.

2 Woher bekommt man die Koordinaten?

Es gibt genau solche Datenbanken, und die wichtigsten sind kostenlos. Die Daten liegen meist im Format **GeoJSON** (eine Textdatei) oder als **Shapefile** (ein Format aus Geoinformationssystemen) vor. Die wichtigsten Quellen:

Quelle	Inhalt	Lizenz
Natural Earth	Staaten, Bundesländer und Provinzen weltweit, Flüsse, Seen, Küsten, Gebirge, Straßen, Städte; drei Detailstufen (1 : 10 Mio., 1 : 50 Mio., 1 : 110 Mio.). *Erste Wahl für Unterrichtskarten.*	gemeinfrei
BKG (VG250, VG2500)	amtliche Verwaltungsgrenzen Deutschlands bis zur Gemeinde, vom Bundesamt für Kartographie und Geodäsie	DL-DE/BY-2.0
OpenStreetMap	sehr detailliert: Grenzen, jeder Bach, jeder See, jede Straße	ODbL
GADM	Verwaltungsgrenzen aller Staaten bis zur Kreisebene	nur nicht-kommerziell

Eine ausführliche, nach Themen sortierte Liste mit Links, von Gebirgen über Autobahnen bis zu den Grenzen des Römischen Reichs, steht im Datenquellen-Katalog in Kapitel 13.

Natural Earth gibt es auf `https://www.naturalearthdata.com`, die GeoJSON-Fassung auf GitHub unter `https://github.com/nvkelso/natural-earth-vector` (Ordner `geojson`). Ein GeoJSON-Eintrag ist gut lesbar. Gekürzt sieht Sachsen so aus; man erkennt den Polygonzug, allerdings in (Längengrad, Breitengrad):

```
{ "type": "Feature",
  "properties": { "name": "Sachsen", "iso_3166_2": "DE-SN", ... },
  "geometry": { "type": "Polygon",
    "coordinates": [[ [14.40, 51.01], [14.39, 51.01], [14.38, 51.02], ... ]] } }
```

> **Lizenz:** Bei Natural Earth ist keine Quellenangabe vorgeschrieben, sie ist aber guter Stil („Kartendaten: Natural Earth“). Bei BKG-Daten und OpenStreetMap ist die Namensnennung *Pflicht*, zum Beispiel „© GeoBasis-DE / BKG 2026“ bzw. „© OpenStreetMap-Mitwirkende“.

3 Vom Globus aufs Papier: die Projektion

Längen- und Breitengrade sind *Winkel* auf einer Kugel. Würde man sie direkt als x- und y-Koordinate zeichnen, wäre Deutschland in Ost-West-Richtung stark gestreckt, denn ein Längengrad ist bei uns deutlich kürzer als am Äquator. Die einfachste Korrektur, die „Plattkarte“, lautet für eine Bezugsbreite φ_0:

$$x = R \cdot \lambda \cdot \cos\varphi_0, \qquad y = R \cdot \varphi \qquad (\lambda, \varphi \text{ im Bogenmaß}, \; R \approx 6371\,\text{km}).$$

Für Deutschland ($\varphi_0 \approx 51°$) ist $\cos\varphi_0 \approx 0{,}63$: Ein Längengrad ist hier also nur etwa 70 km lang, ein Breitengrad etwa 111 km. Für saubere Karten nimmt man eine fertige Projektion aus der Python-Bibliothek `pyproj`. Jede Projektion hat einen Code:

Gebiet	**Projektion**	**Code**
Deutschland (West)	UTM Zone 32N (amtlich)	EPSG:25832
Ostdeutschland, Sachsen	UTM Zone 33N	EPSG:25833
Europa	Lambert, flächentreu	EPSG:3035
USA (48 Staaten)	Albers, flächentreu	EPSG:5070
Alaska / Hawaii	Alaska Albers / Hawaii Albers	EPSG:3338 / ESRI:102007
Welt, flächentreu	Equal Earth	EPSG:8857
Welt, Kompromiss	Robinson	ESRI:54030

Wie stark Projektionen sich unterscheiden, zeigt Kapitel 9.

4 Vereinfachen: der Douglas-Peucker-Algorithmus

Die Natural-Earth-Grenze von Sachsen hat 645 Punkte. Auf einer 5 cm breiten Karte liegen viele davon weniger als 0,1 mm auseinander; sie kosten nur Kompilierzeit. Der **Douglas-Peucker-Algorithmus** entfernt solche Punkte:

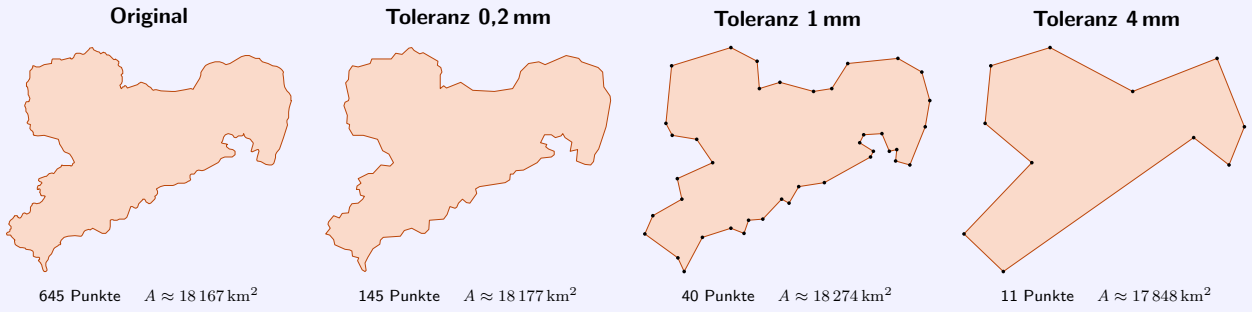
1. Verbinde Anfangs- und Endpunkt des Linienzugs durch eine Strecke.
2. Suche den Punkt mit dem größten Abstand zu dieser Strecke.
3. Ist dieser Abstand kleiner als die **Toleranz** ε, werden alle Zwischenpunkte gelöscht. Sonst wird der Punkt behalten und das Verfahren *rekursiv* auf beide Teilstücke angewendet.

Die Datei `sachsen_stufen.tex` enthält Sachsen in vier Stufen, jeweils mit Punktzahl und Fläche. Die Karte zeigt sie nebeneinander, bei den beiden groben Stufen mit allen Eckpunkten:

```
% Präambel: \input{sachsen_stufen.tex}
\def\zeigeinfo#1/#2\relax{#1 Punkte \quad $A \approx #2\,\mathrm{km}^2$}
\newcommand\info[1]{\expandafter\expandafter\expandafter
  \zeigeinfo\csname geo:info:SN-#1\endcsname\relax}
\begin{tikzpicture}[font=\sffamily\scriptsize,
                    umriss/.style={fill=akzentorange!22, draw=akzentorange!80!black,
                                   line width=0.5pt, line join=round}]
  \foreach \stufe/\x/\titel in {Original/0/Original, Fein/5.6/{Toleranz 0,2\,mm},
                                Mittel/11.2/{Toleranz 1\,mm}, Grob/16.8/{Toleranz 4\,mm}} {
    \begin{scope}[xshift=\x cm]
```

```
      \path[umriss] \csname geo:gebiet:SN-\stufe\endcsname;
      \node[font=\sffamily\small\bfseries] at (2.5,4.45) {\titel};
      \node at (2.5,-0.4) {\info{\stufe}};
    \end{scope}
  }
  % Eckpunkte der beiden groben Stufen sichtbar machen
  \foreach \stufe/\x in {Mittel/11.2, Grob/16.8} {
    \expandafter\let\expandafter\liste\csname geo:punkte:SN-\stufe\endcsname
    \begin{scope}[xshift=\x cm]
      \foreach \p in \liste { \fill[black] \p circle (0.9pt); }
    \end{scope}
  }
\end{tikzpicture}
```

Ausgabe



Bei 0,2 mm Toleranz bleiben nur 145 der 645 Punkte übrig, ohne dass man einen Unterschied sieht. Die Flächen wurden mit der Gaußschen Trapezformel berechnet (siehe Kapitel 12). Amtlich ist Sachsen rund 18 450 km² groß; auch die „Original“-Daten sind also schon leicht vereinfacht.

> **Gemeinsame Grenzen:** Vereinfacht man jedes Bundesland *einzeln*, werden die gemeinsamen Grenzen unterschiedlich gekürzt, und es entstehen Lücken oder Überlappungen. Das Skript `karten_beispiele.py` nutzt deshalb `shapely.coverage_simplify`, das alle Länder gemeinsam vereinfacht; jede Grenze wird dabei nur einmal bearbeitet. Beim allgemeinen Werkzeug `geo2tikz.py` hilft eine kleine Toleranz (Standard 0,02 cm).

5 Zwei Werkzeuge

Im Ordner `scripts` des Skills liegen zwei Python-Skripte. Beide lesen GeoJSON, projizieren, vereinfachen und schreiben eine `.tex`-Datei; sie unterscheiden sich im Zweck:

	`geo2tikz.py`	`karten_beispiele.py`
Zweck	eigene Karten mit beliebigem Ausschnitt	der fertige Atlas dieser Handreichung
Einstellungen	alles über die Kommandozeile: Ausschnitt, Breite, Projektion, Ebenen	fest im Skript, je eine Funktion pro Karte
Zeichnen mit	`\geolaender`, Stile `geo laender/DE-SN`	`\Gebiet{DE-SN}`, `\Fluss{Elbe}`, …
Besonderheiten	jede GeoJSON-Datei, jeder Filter	Höhenschichten, Gebirge, Autobahnen, Einschübe (USA), Färbung, Tissot-Kreise, historische Daten
Kapitel	6	7 bis 10

Beide brauchen Python 3 mit einigen Zusatzpaketen, die man einmalig installiert:

```
pip install shapely pyproj                # geo2tikz.py
pip install shapely pyproj h5py contourpy scipy  # karten_beispiele.py (Höhenschichten)
```

Für die Karten selbst genügt Ti*k*Z und das ist in der universalen `header.tex` bereits geladen.

6 Eigene Karten mit `geo2tikz.py`

6.1 Daten herunterladen

Im Terminal lädt `curl` die Dateien direkt herunter. Für eine Deutschlandkarte mit Bundesländern, Nachbarstaaten, Flüssen, Seen und Städten:

```
B=https://raw.githubusercontent.com/nvkelso/natural-earth-vector/master/geojson
curl -sSfLO $B/ne_10m_admin_1_states_provinces.geojson
curl -sSfLO $B/ne_50m_admin_0_countries.geojson
curl -sSfLO $B/ne_10m_rivers_lake_centerlines.geojson
curl -sSfLO $B/ne_10m_lakes.geojson
curl -sSfLO $B/ne_10m_populated_places_simple.geojson
```

Die Zahl im Dateinamen gibt die Genauigkeit an: `10m` ist der feinste Maßstab (1 : 10 Mio.), `50m` und `110m` sind gröber und reichen für Übersichtskarten von Europa oder der Welt. Die Rohdaten sind teilweise sehr groß (die Bundesländer der ganzen Welt umfassen 39 MB); man legt sie nicht in Materialsammlungen ab, sondern notiert den Download-Befehl.

6.2 Der Aufruf

Das Skript bekommt den Kartenausschnitt in Grad, die gewünschte Breite in Zentimetern, eine Projektion und eine oder mehrere Ebenen. Alle Ebenen einer Karte werden in **einem** Aufruf umgerechnet, sonst haben sie verschiedene Maßstäbe, und die Flüsse liegen neben den Ländern. Mit diesem Aufruf entstand die Datei für dieses Kapitel:

```
python3 geo2tikz.py --bbox 5.8,47.2,15.1,55.1 --breite 9 --proj EPSG:3035 --toleranz 0.03 \
  --ebene "name=laender; datei=ne_10m_admin_1_states_provinces.geojson; filter=iso_a2=DE; id=iso
      _3166_2" \
  --ebene "name=nachbarn; datei=ne_50m_admin_0_countries.geojson; filter=ISO_A2_EH=PL|CZ|AT|CH|FR
      |BE|NL|LU|DK; id=ISO_A2_EH" \
  --ebene "name=fluesse; datei=ne_10m_rivers_lake_centerlines.geojson; id=name" \
  --ebene "name=seen; datei=ne_10m_lakes.geojson; id=name" \
  --ebene "name=staedte; datei=ne_10m_populated_places_simple.geojson; filter=adm0_a3=DEU; id=
      name" \
  -o beispiel_deutschland.tex
```

Das Skript meldet danach, wie viele Objekte jede Ebene enthält und wie groß die Karte wird, hier $9{,}0 \times 11{,}4$ cm.

Option	Bedeutung
`--bbox`	Ausschnitt als Länge/Breite links unten und rechts oben in Grad. Bei negativen Werten mit Gleichheitszeichen schreiben: `--bbox=-180,-58,180,84`
`--breite`	Kartenbreite in cm; die Höhe ergibt sich aus der Projektion
`--proj`	`plattkarte` (Standard), `mercator` oder ein Code aus Kapitel 3, z. B. `EPSG:3035`
`--toleranz`	Vereinfachung der Umrisse in cm (Kapitel 4); `0` schaltet sie aus
`--ebene`	eine Ebene: `name` (nur Buchstaben), `datei`, optional `filter` und `id`
`-o`	Name der erzeugten Datei

Zwei Fallstricke bei den Filtern: Bundesländer haben in Natural Earth die Kennung `iso_3166_2` (`DE-SN`, `DE-BY` …). Bei Staaten fehlen Frankreich und Norwegen, wenn man nach `ISO_A2` filtert, weil dort `-99` eingetragen ist. Deshalb filtert man Staaten nach `ISO_A2_EH` oder `ADM0_A3` (`DEU`, `FRA`).

6.3 Was in der erzeugten Datei steht

Für jede Ebene schreibt das Skript einen Befehl, der alle Objekte zeichnet, und einen leeren Stil für die Ebene und für jedes einzelne Objekt. Außerdem legt es für jedes Objekt einen Punkt an, der garantiert im Inneren liegt. Gekürzt:

```
\tikzset{geo laender/.style={},
  geo laender/DE-SN/.style={},
  geo laender/DE-BY/.style={}, ...}
\newcommand\geolaender{%
\path[geo laender, geo laender/DE-SN, even odd rule] (8.08,5.57) -- (7.90,5.56) -- ... -- cycle;
...
\coordinate (laender-DE-SN) at (7.17,5.32);
...}
```

Element	Beispiel	Zweck
Befehl der Ebene	`\geolaender`	zeichnet alle Objekte der Ebene
Stil der Ebene	`geo laender/.style={...}`	Aussehen aller Objekte
Stil eines Objekts	`geo laender/DE-SN/.style={...}`	ein Objekt hervorheben
Punkt im Objekt	`(laender-DE-SN)`	Beschriftung, Markierung

Weil alle Stile zunächst leer sind, zeichnet ein Befehl wie `\geolaender` ohne eigenen Stil nichts. Erst ein Stil mit `fill` oder `draw` macht die Ebene sichtbar.

6.4 Die erste Karte

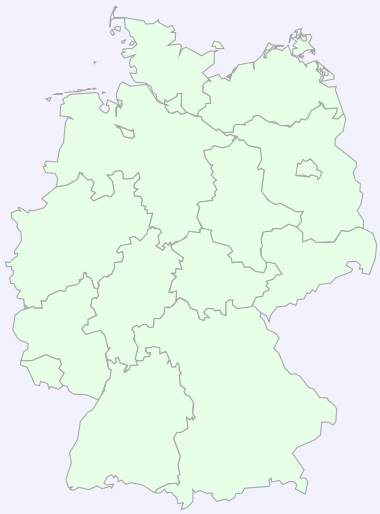
Die erzeugte Datei wird in der Präambel geladen:

```
\input{beispiel_deutschland.tex}
```

Danach genügen ein Stil für die Ebene und der Befehl `\geolaender`. Mit `scale` lässt sich die Karte verkleinern:

```
\begin{tikzpicture}[scale=0.6,
  geo laender/.style={fill=green!10, draw=gray!70, line width=0.4pt}]
  \geolaender
\end{tikzpicture}
```

Ausgabe



Die Stile stehen hier in den Optionen der `tikzpicture`. Sie gelten dann nur für diese eine Karte, und dieselben Daten lassen sich im selben Dokument für eine zweite Karte mit anderem Aussehen verwenden.

6.5 Mehrere Ebenen, Hervorhebungen und Städte

Die Ebenen werden von unten nach oben gezeichnet: erst die Nachbarstaaten, dann die Bundesländer, darüber Seen und Flüsse und zuletzt die Beschriftung. `\clip` schneidet alles am Kartenrand ab; die Maße stehen in der letzten Zeile der erzeugten Datei. Die Ebene `staedte` zeichnet selbst nichts, legt aber die Punkte der Städte an.

```
\begin{tikzpicture}[scale=0.85,
  geo nachbarn/.style={fill=gray!12, draw=gray!50, line width=0.3pt},
  geo laender/.style={fill=green!8, draw=gray!70, line width=0.4pt},
  geo laender/DE-SN/.style={fill=orange!45},
  geo seen/.style={fill=blue!25, draw=blue!50, line width=0.3pt},
  geo fluesse/.style={draw=blue!60, line width=0.6pt, line cap=round, line join=round}]
  \clip (0,0) rectangle (9,11.37);
  \geonachbarn \geolaender \geoseen \geofluesse \geostaedte
  \foreach \s/\n/\pos in {Berlin/Berlin/right, Hamburg/Hamburg/left, Leipzig/Leipzig/left,
                          Dresden/Dresden/right, Munich/München/right, Cologne/Köln/left}
    \fill (staedte-\s) circle (1.5pt) node[\pos, font=\scriptsize] {\n};
  \node[below right, font=\scriptsize\itshape, text=blue!70!black] at (fluesse-Elbe) {Elbe};
  \node[anchor=south east, font=\tiny, fill=white, inner sep=1pt] at (9,0) {Kartendaten: Natural
      Earth (gemeinfrei)};
\end{tikzpicture}
```

Ausgabe



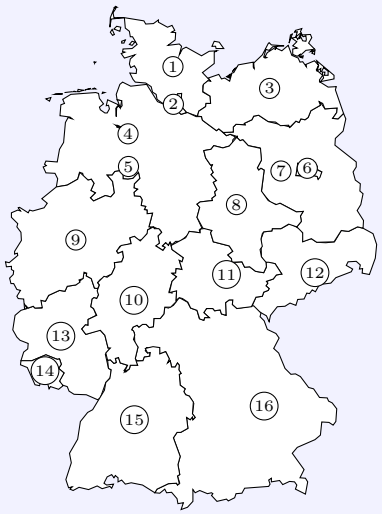
Die Namen der Städte stammen aus Natural Earth und sind englisch (`Munich`, `Cologne`). Umlaute ersetzt das Skript in den Namen der Punkte, aus Düsseldorf wird `staedte-Duesseldorf`. Welche Namen es gibt, zeigt ein Blick in die erzeugte Datei. Bei Flüssen lohnt das besonders: Der Rhein heißt je nach Abschnitt `Rhein`, `Rhin` oder `Rhine`.

6.6 Eine stumme Karte

Für ein Arbeitsblatt bekommen die Bundesländer Nummern statt Namen. Die Schleife setzt an den Punkt jedes Landes einen kleinen Kreis mit seiner Nummer:

```
\begin{tikzpicture}[scale=0.6,
  geo laender/.style={fill=white, draw=black, line width=0.4pt}]
  \geolaender
  \foreach \l [count=\n] in {SH,HH,MV,HB,NI,BE,BB,ST,NW,HE,TH,SN,RP,SL,BW,BY}
    \node[circle, fill=white, draw, inner sep=0.8pt, font=\tiny] at (laender-DE-\l) {\n};
\end{tikzpicture}
```

Ausgabe



Die Lösung ergibt sich aus der Reihenfolge in der Liste: 1 Schleswig-Holstein, 2 Hamburg, 3 Mecklenburg-Vorpommern, 4 Bremen und so weiter. Wer die Reihenfolge der Liste ändert, bekommt eine neue Fassung für die Parallelklasse.

7 Der Beispiel-Atlas

Die Karten der folgenden Kapitel liegen als fertige Vorlagen im Ordner `assets` des Skills (`karte_*.tex`), zusammen mit ihren Datendateien (`*_daten.tex`). Jede Vorlage ist ein eigenständiges `standalone`-Dokument und lässt sich mit `pdflatex` direkt übersetzen. Die Datendateien hat das Skript `karten_beispiele.py` erzeugt; wer sie neu erzeugen oder verändern will:

```
python3 karten_beispiele.py download      # Rohdaten laden (ca. 170 MB, einmalig)
python3 karten_beispiele.py deutschland   # -> deutschland_daten.tex
python3 karten_beispiele.py usa           # -> usa_daten.tex
python3 karten_beispiele.py sachsen       # -> sachsen_stufen.tex
python3 karten_beispiele.py europa        # -> europa_daten.tex
python3 karten_beispiele.py welt          # -> welt_mercator/_robinson/_equalearth_daten.tex
python3 karten_beispiele.py rom           # -> rom_daten.tex
python3 karten_beispiele.py historisch    # -> historisch_daten.tex
python3 karten_beispiele.py               # alles auf einmal
```

Die wichtigsten Stellschrauben stehen am Anfang der jeweiligen Funktion im Skript, zum Beispiel in `deutschland()`:

```
bbox = (5.4, 47.1, 15.6, 55.15)           # Kartenausschnitt in Grad
k = Karte("EPSG:25832", bbox, breite_cm=12, toleranz_cm=0.025)
```

7.1 Die Befehle der Datendateien

Eine Datendatei wie `deutschland_daten.tex` wird mit `\input` geladen und stellt diese Befehle bereit:

Befehl	Bedeutung
`\Gebiet{DE-SN}`	Polygonzug eines Gebiets (Bundesländer nach ISO 3166-2: `DE-BY`, `DE-NW`, …; Nachbarstaaten `POL`, `CZE`, …; ganz Deutschland `DEU`; US-Staaten `TX`, `CA`, …)
`\Fluss{Elbe}`	Linienzug eines Flusses
`\See{Bodensee}`	Polygonzug eines Sees
`\GeoKoordinaten`	legt benannte Punkte an: die Mitte jedes Gebiets, z. B. `(DE-SN)`, und Städte, z. B. `(Leipzig)`
`\Rahmen`	Rechteck des Kartenausschnitts (zum Clippen)
`\HundertKm`	Länge von 100 km auf der Karte in cm (Maßstabsleiste)
`\GebietListe`	alle Gebietskennungen der Datei, z. B. für `\foreach \g in \GebietListe`
`\Hoehe{500}`	Höhenschicht: alle Flächen höher als 500 m
`\Gebirge{Harz}`	Fläche eines Gebirges
`\Autobahn{A9}`	Linienzug einer Autobahn
`\Bahn{alle}`	alle Bahnstrecken
`\...Liste`	zu jeder Ebene die Liste ihrer Namen, z. B. `\AutobahnListe`, `\HoeheListe`

Intern ist jedes Gebiet ein Makro mit dem Polygonzug. Inseln, etwa Rügen, stehen als weitere Teilpfade im selben Makro, deshalb genügt ein einziges `\fill \Gebiet{DE-MV};`.

```
\expandafter\def\csname geo:gebiet:DE-SN\endcsname{
  (10.09,6.96) -- (10.08,6.97) -- ... -- (10.11,6.96) -- cycle}
```

Merke:

Städte und Ländermitten erst mit `\GeoKoordinaten` anlegen, dann wie gewohnt verwenden: `\fill (Leipzig) circle (1pt);` oder `\node at ($(Berlin)+(0.5,0)$) {BE};`. Die Mitte wird als „Pol der Unzugänglichkeit“ berechnet, also als der Punkt, der am weitesten vom Rand entfernt ist. So landen Beschriftungen sicher im Land.

In den folgenden Beispielen steht über dem Code, welche Datendatei in der Präambel geladen wird. Die Ausgabe ist jeweils auf die Zeilenbreite verkleinert, wenn die Karte breiter ist.

7.2 Lagekarte: Sachsen in Deutschland

Alle Länder hellgrau, Sachsen farbig, dazu Elbe, Saale und drei Städte. Die Vorlage heißt `karte_sachsen_lage.tex`.

```
% Präambel: \input{deutschland_daten.tex}
\colorlet{wasser}{akzentblau!55!cyan!80!black}
\begin{tikzpicture}[scale=0.6]
  \GeoKoordinaten

  % alle Länder hellgrau, Sachsen farbig
  \foreach \land in {SH,HH,NI,HB,MV,BB,BE,ST,TH,HE,NW,RP,SL,BW,BY}
    \fill[black!10, draw=white, line width=0.4pt] \Gebiet{DE-\land};
  \fill[akzentorange!35, draw=akzentorange!80!black, line width=0.6pt] \Gebiet{DE-SN};
  \draw[black!60, line width=0.6pt] \Gebiet{DEU};

  % Elbe und Saale
  \draw[wasser, line width=0.5pt] \Fluss{Elbe} \Fluss{Saale};

  % Städte
  \foreach \stadt/\lage in {Leipzig/left, Dresden/above right, Chemnitz/below}
    \fill (\stadt) circle (1.3pt) node[\lage, font=\sffamily\tiny] {\stadt};
\end{tikzpicture}
```

Ausgabe



7.3 Deutschland mit Bundesländern, Flüssen und Seen

Die große Karte (`karte_deutschland_bundeslaender.tex`) zeichnet in Schichten von hinten nach vorn: Meer, Nachbarstaaten, Bundesländer, Bundesgrenze, Flüsse, Seen, Städte, Beschriftung, Maßstabsleiste und Nordpfeil. Die Bundesländer sind mit vier Farben so gefärbt, dass benachbarte Länder nie dieselbe Farbe haben (Vierfarbensatz). Kleine Länder bekommen ihr Kürzel mit einer Hinweislinie.

```
% Präambel: \input{deutschland_daten.tex}
\colorlet{farbeA}{akzentblau!18}
\colorlet{farbeB}{akzentgruen!18}
\colorlet{farbeC}{akzentorange!20}
\colorlet{farbeD}{akzentlila!18}
\colorlet{wasser}{akzentblau!55!cyan!80!black}
\colorlet{meer}{akzentblau!7}
\begin{tikzpicture}[
    land/.style    = {draw=white, line width=0.6pt, line join=round},
    fluss/.style   = {draw=wasser, line width=0.55pt, line join=round, line cap=round},
    kuerzel/.style = {font=\sffamily\scriptsize\bfseries, text=black!75},
  ]

  % --- Hintergrund: Meer und Nachbarstaaten ---------------------------
  \clip \Rahmen;
  \fill[meer] \Rahmen;
  \foreach \staat in {DNK,POL,CZE,AUT,CHE,LIE,FRA,LUX,BEL,NLD}
    \fill[black!7, draw=white, line width=0.8pt] \Gebiet{\staat};

  % --- Bundesländer, eingefärbt nach dem Vierfarbensatz ----------------
  \foreach \land/\farbe in {NI/A, SH/B, HH/C, MV/C, HB/B, BB/B, BE/A, ST/C,
                            SN/A, TH/B, HE/C, NW/B, RP/A, SL/B, BW/B, BY/D}
    \fill[farbe\farbe, land] \Gebiet{DE-\land};

  % --- Bundesgrenze kräftig nachziehen ---------------------------------
  \draw[black!65, line width=0.9pt, line join=round] \Gebiet{DEU};

  % --- Gewässer --------------------------------------------------------
  \foreach \f in {Rhein,Elbe,Donau,Weser,Main,Oder,Mosel,Ems,Inn,Neckar,
                  Saale,Spree,Isar,Havel,Fulda,Werra,Aller,Lech,Neisse,
                  WeisseElster,Mulde,Lahn,Ruhr}
    \draw[fluss] \Fluss{\f};
  \foreach \s in {Bodensee,Mueritz,SchwerinerSee,Chiemsee,StarnbergerSee,
                  Ammersee,SteinhuderMeer,PlauerSee,KummerowerSee}
    \fill[wasser] \See{\s};

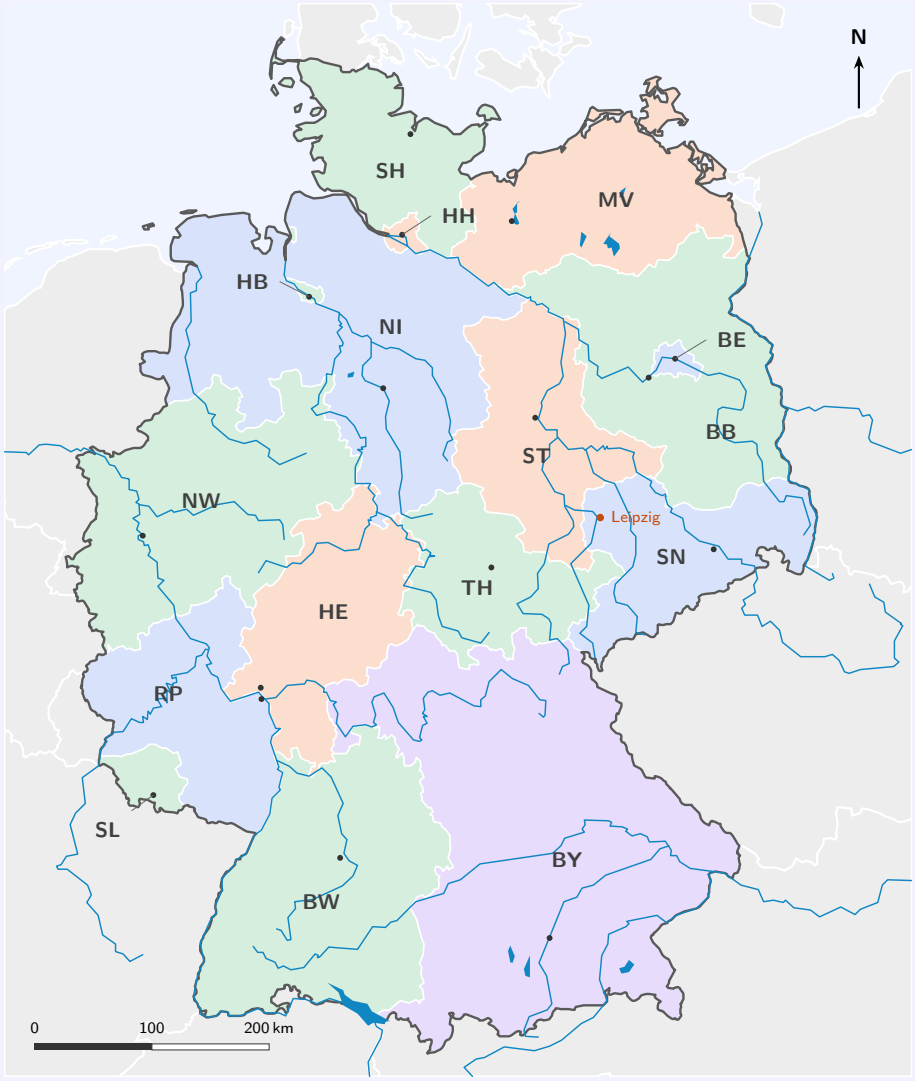
  % --- Landeshauptstädte -----------------------------------------------
  %     \GeoKoordinaten legt alle Orte und Ländermitten als benannte
  %     Punkte an: (Berlin), (DE-SN), ($(Berlin)+(1,0)$) usw.
  \GeoKoordinaten
  \foreach \stadt in {Kiel,Hamburg,Schwerin,Bremen,Hannover,Berlin,Potsdam,
                      Magdeburg,Duesseldorf,Erfurt,Dresden,Wiesbaden,Mainz,
                      Saarbruecken,Stuttgart,Muenchen}
    \fill[black!80] (\stadt) circle (1.1pt);
  \fill[akzentorange!85!black] (Leipzig) circle (1.4pt)
    node[right, font=\sffamily\tiny] {Leipzig};

  % --- Kürzel der Länder (kleine Länder mit Hinweislinie) --------------
  \foreach \land in {NI,SH,MV,BB,ST,SN,TH,HE,NW,RP,BW,BY}
    \node[kuerzel] at (DE-\land) {\land};
  \node[kuerzel] (hh) at ($(Hamburg)+(0.75,0.25)$) {HH};
  \node[kuerzel] (hb) at ($(Bremen)+(-0.75,0.2)$) {HB};
  \node[kuerzel] (be) at ($(Berlin)+(0.75,0.25)$) {BE};
  \node[kuerzel] (sl) at ($(Saarbruecken)+(-0.6,-0.45)$) {SL};
  \draw[black!60, line width=0.3pt] (Hamburg) -- (hh.west)
                                    (Bremen)  -- (hb.east)
                                    (Berlin)  -- (be.west)
                                    (Saarbruecken) -- (sl.north east);

  % --- Maßstabsleiste und Nordpfeil ------------------------------------
  \begin{scope}[shift={(0.4,0.35)}, font=\sffamily\tiny]
    \fill[black!80] (0,0) rectangle (\HundertKm,0.08);
    \fill[white, draw=black!80, line width=0.3pt] (\HundertKm,0) rectangle (2*\HundertKm,0.08);
```

```
    \node[above] at (0,0.08) {0};
    \node[above] at (\HundertKm,0.08) {100};
    \node[above] at (2*\HundertKm,0.08) {200\,km};
  \end{scope}
  \draw[-{Stealth[length=5pt]}, line width=0.8pt] (11.3,12.8) -- (11.3,13.5)
    node[above, font=\sffamily\scriptsize\bfseries] {N};
\end{tikzpicture}
```

Ausgabe



7.4 USA mit Bundesstaaten

Für die USA wird die flächentreue Albers-Projektion benutzt. Alaska und Hawaii sind, wie in Atlanten üblich, mit eigener Projektion und eigenem Maßstab als Einschub unten links eingesetzt. Die Farben zeigen die vier Census-Regionen.

```
% Präambel: \input{usa_daten.tex}
\colorlet{wasser}{akzentblau!55!cyan!80!black}
\begin{tikzpicture}[
    staat/.style    = {draw=white, line width=0.5pt, line join=round},
    fluss/.style    = {draw=wasser, line width=0.5pt, line join=round, line cap=round},
    kuerzel/.style = {font=\sffamily\tiny\bfseries, text=black!70},
```

```
  ]
  \GeoKoordinaten

  % --- Staaten: Farbe nach Region (Census Regions) ---------------------
  \foreach \s in {ME,NH,VT,MA,RI,CT,NY,NJ,PA}
    \fill[akzentblau!20, staat] \Gebiet{\s};                    % Nordosten
  \foreach \s in {OH,MI,IN,IL,WI,MN,IA,MO,ND,SD,NE,KS}
    \fill[akzentgruen!20, staat] \Gebiet{\s};                   % Mittlerer Westen
  \foreach \s in {DE,MD,DC,VA,WV,NC,SC,GA,FL,KY,TN,AL,MS,AR,LA,OK,TX}
    \fill[akzentorange!22, staat] \Gebiet{\s};                  % Süden
  \foreach \s in {MT,ID,WY,CO,NM,AZ,UT,NV,CA,OR,WA,AK,HI}
    \fill[akzentlila!18, staat] \Gebiet{\s};                  % Westen

  % --- Gewässer ---------------------------------------------------------
  \foreach \f in {Mississippi,Missouri,Ohio,Colorado,RioGrande,Columbia,
                  Arkansas,Tennessee,Snake}
    \draw[fluss] \Fluss{\f};
  \foreach \s in {Superior,Michigan,Huron,Erie,Ontario,GreatSaltLake}
    \fill[wasser] \See{\s};

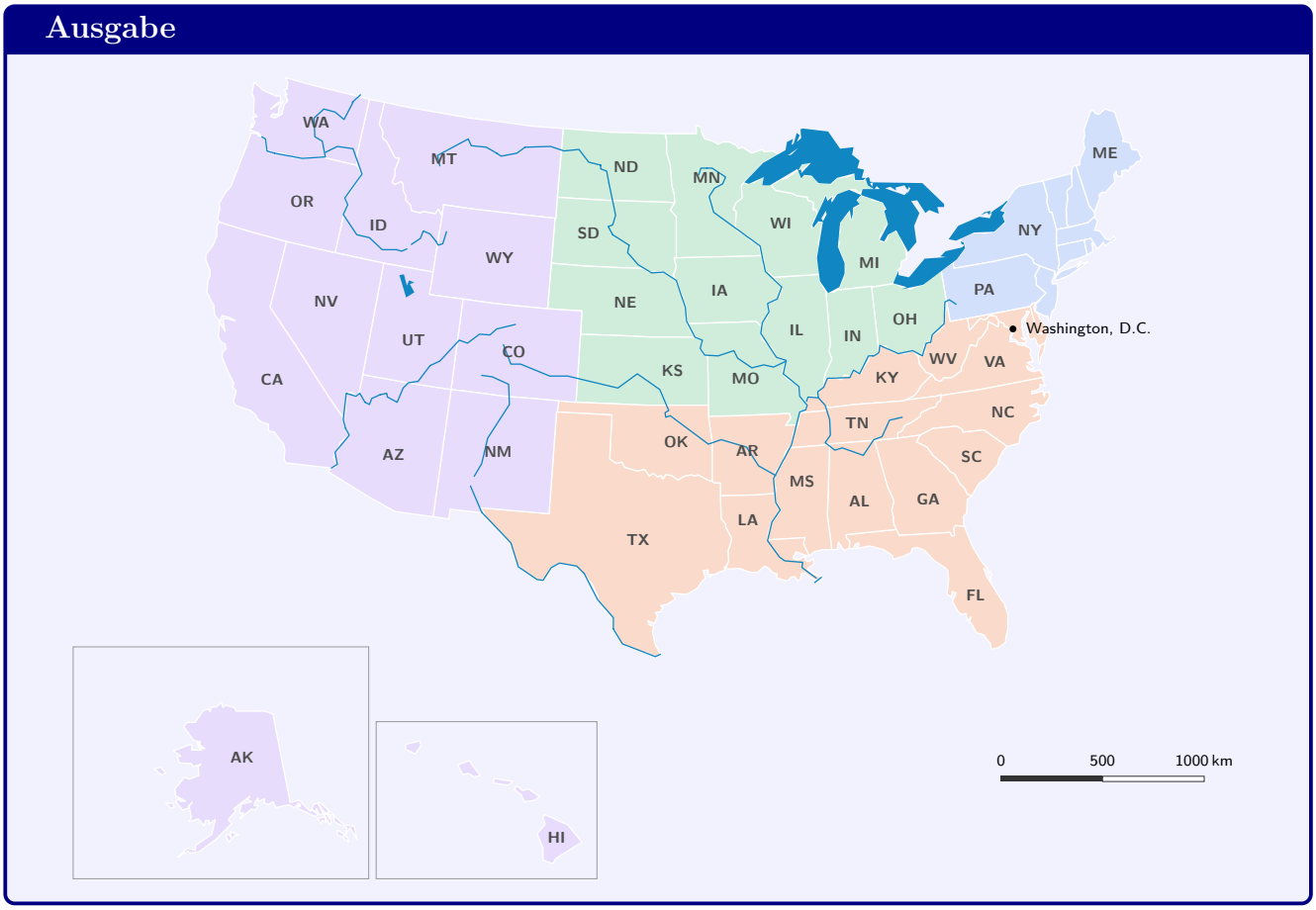
  % --- Kürzel (nur Staaten, die groß genug sind) ------------------------
  \foreach \s in {WA,OR,CA,NV,ID,MT,WY,UT,AZ,CO,NM,ND,SD,NE,KS,OK,TX,MN,IA,
                  MO,AR,LA,WI,IL,MI,IN,OH,KY,TN,MS,AL,GA,FL,SC,NC,VA,WV,PA,
                  NY,ME,AK,HI}
    \node[kuerzel] at (\s) {\s};

  % --- Hauptstadt -------------------------------------------------------
  \fill[black] (Washington) circle (1.3pt)
    node[right=1pt, font=\sffamily\tiny] {Washington, D.C.};

  % --- Rahmen um die Einschübe ------------------------------------------
  \draw[black!40, line width=0.3pt] (-0.2,-2.5) rectangle (3.75,0.6)
                                    (3.85,-2.5) rectangle (6.8,-0.4);

  % --- Maßstabsleiste (gilt nur für die 48 Staaten) ---------------------
  \begin{scope}[shift={(12.2,-1.2)}, font=\sffamily\tiny]
    \fill[black!80] (0,0) rectangle (5*\HundertKm,0.07);
    \fill[white, draw=black!80, line width=0.3pt] (5*\HundertKm,0) rectangle (10*\HundertKm,0.07)
        ;
    \node[above] at (0,0.07) {0};
    \node[above] at (5*\HundertKm,0.07) {500};
    \node[above] at (10*\HundertKm,0.07) {1000\,km};
  \end{scope}
\end{tikzpicture}
```

Ausgabe



8 Mehr Ebenen für Deutschland: Gebirge, Höhen, Verkehr

Natural Earth enthält weit mehr als Grenzen und Flüsse. Für Deutschland sind unter anderem verfügbar:

- **Gebirge** als Flächen (Harz, Erzgebirge, Böhmerwald, Alpen, Vogesen, Ardennen, Jura, Sudeten) und **Landschaften** wie das Norddeutsche Tiefland,
- **Gipfel** mit Lage und Höhe (Zugspitze, Feldberg, Brocken, Hohe Acht; der Fichtelberg fehlt in Natural Earth und wurde im Skript ergänzt),
- **Straßen** mit Nummern (alle großen Autobahnen, z. B. „A9“), **Bahnstrecken**, Flughäfen und Häfen,
- außerdem Städte mit Einwohnerzahl, Zeitzonen, Meerestiefen, Gradnetz und vieles mehr.

Das Skript legt dafür zusätzliche **Ebenen** an. Jede Ebene bekommt einen eigenen Befehl und eine Liste ihrer Namen, zum Beispiel `\Hoehe{500}` und `\HoeheListe`.

8.1 Höhenschichten aus einem Höhenmodell

Für das Relief braucht man ein **digitales Höhenmodell**: ein Gitter, in dem für jeden Punkt (x, y) die Höhe $z = f(x, y)$ gespeichert ist. Das Skript berechnet daraus für jede Stufe h die Fläche $\{(x, y) \mid f(x, y) \geq h\}$, mathematisch eine **Niveaumenge**; ihr Rand ist die **Höhenlinie** (Niveaulinie) zur Höhe h. Gezeichnet werden die Flächen von unten nach oben übereinander, wie die Farbstufen im Atlas. Intern arbeitet dabei der Algorithmus *Marching Squares* (Bibliothek `contourpy`): Er prüft jedes Gitterquadrat, ob die Höhenlinie hindurchläuft, und setzt die Linie stückweise zusammen.

```
% Präambel: \input{deutschland_daten.tex}
\colorlet{wasser}{akzentblau!55!cyan!80!black}
\colorlet{meer}{akzentblau!7}
\definecolor{h0}{HTML}{BFE0B0}   % unter 100 m
\definecolor{h100}{HTML}{DCEBB5}
\definecolor{h200}{HTML}{EEEDB4}
\definecolor{h300}{HTML}{EDDDA2}
\definecolor{h500}{HTML}{E4C58D}
\definecolor{h750}{HTML}{D4A877}
\definecolor{h1000}{HTML}{C08D63}
\definecolor{h1500}{HTML}{A6735A}
\begin{tikzpicture}[
    fluss/.style  = {draw=wasser, line width=0.5pt, line join=round, line cap=round},
    gebirge/.style = {font=\sffamily\scriptsize\itshape, text=black!75},
  ]
  \GeoKoordinaten
  \clip \Rahmen;
  \fill[meer] \Rahmen;
  \foreach \staat in {DNK,POL,CZE,AUT,CHE,LIE,FRA,LUX,BEL,NLD}
    \fill[black!7, draw=white, line width=0.8pt] \Gebiet{\staat};

  % --- Höhenschichten: von unten nach oben übereinander malen -----------
  \fill[h0] \Gebiet{DEU};
  \foreach \h in \HoeheListe
    \fill[h\h] \Hoehe{\h};

  % --- Länder- und Bundesgrenzen, Gewässer ------------------------------
  \foreach \land in {SH,HH,NI,HB,MV,BB,BE,ST,SN,TH,HE,NW,RP,SL,BW,BY}
    \draw[white, line width=0.35pt, line join=round] \Gebiet{DE-\land};
  \draw[black!65, line width=0.9pt, line join=round] \Gebiet{DEU};
  \foreach \f in {Rhein,Elbe,Donau,Weser,Main,Oder,Mosel,Ems,Inn,Neckar,Saale,
                  Spree,Isar,Havel,Fulda,Werra,Aller,Lech,Neisse,WeisseElster,Mulde}
    \draw[fluss] \Fluss{\f};
  \foreach \s in {Bodensee,Mueritz,SchwerinerSee,Chiemsee,StarnbergerSee,Ammersee}
    \fill[wasser] \See{\s};

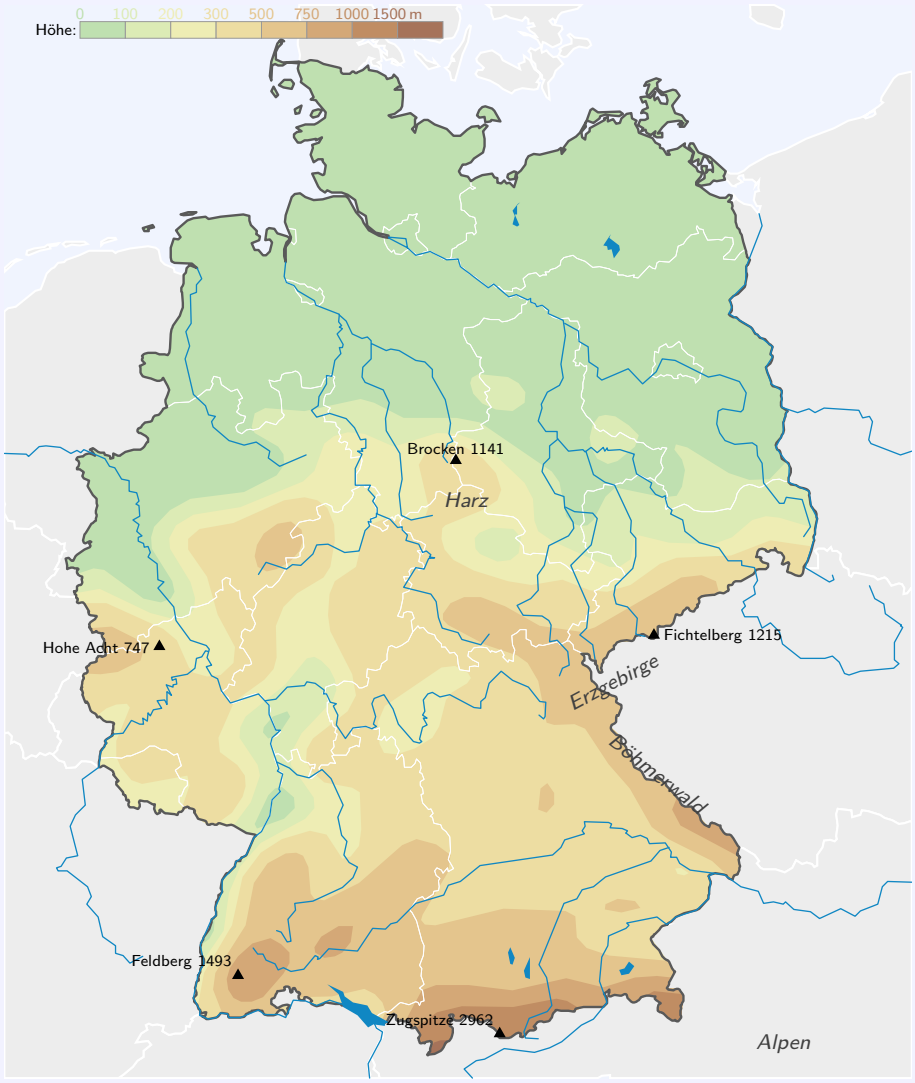
  % --- Gebirgsnamen (nur die in bzw. an Deutschland) --------------------
  \node[gebirge] at ($(Gebirge-Harz)+(0,-0.25)$) {Harz};
  \node[gebirge, rotate=28] at ($(Gebirge-Erzgebirge)+(0.2,-0.3)$) {Erzgebirge};
  \node[gebirge, rotate=-38] at (Gebirge-Boehmerwald) {Böhmerwald};
  \node[gebirge] at (10.3,0.45) {Alpen};

  % --- Gipfel mit Höhe: Dreieck aus der Liste, Beschriftung von Hand ------
  \foreach \g/\name/\h in \GipfelListe
    \fill[black] (\g) ++(0,0.1) -- ++(-0.08,-0.14) -- ++(0.16,0) -- cycle;
  \begin{scope}[font=\sffamily\tiny, inner sep=1.5pt]
    \node[above=1pt]       at (Brocken)     {Brocken 1141};
    \node[right=2pt]       at (Fichtelberg) {Fichtelberg 1215};
    \node[left=2pt]        at (HoheAcht)    {Hohe Acht 747};
    \node[above left=1pt]  at (Feldberg)    {Feldberg 1493};
```

```
    \node[above left=1pt]  at (Zugspitze)   {Zugspitze 2962};
  \end{scope}

  % --- Legende ------------------------------------------------------------
  \begin{scope}[shift={(1.0,13.75)}, font=\sffamily\tiny]
    \foreach \farbe/\text [count=\i] in {h0/{0}, h100/{100}, h200/{200}, h300/{300},
                                         h500/{500}, h750/{750}, h1000/{1000}, h1500/{1500\,m}}
      \fill[\farbe, draw=black!40, line width=0.2pt] (0.6*\i-0.6,0) rectangle ++(0.6,0.22)
        node[above, inner sep=1pt] at (0.6*\i-0.6,0.22) {\text};
    \node[left, inner sep=1pt] at (0,0.11) {Höhe:};
  \end{scope}
\end{tikzpicture}
```

Ausgabe



Auflösung: Für diese Karte wurde ein grobes, weltweites Höhenmodell benutzt (ETOPO1 der NOAA, auf 10 Bogenminuten gemittelt; ein Gitterpunkt etwa alle 12 km × 18 km). Für Deutschland im Ganzen reicht das, aber Gipfel werden „abgeflacht“: Das Erzgebirge erreicht in diesem Raster keine 800 m. Für Detailkarten, etwa das Erzgebirge oder den Harz mit Höhenlinien alle 100 m, nimmt man das **Copernicus-Höhenmodell** mit 30 m Auflösung (Link im Katalog). Das Skript liest GeoTIFF-Dateien direkt (dafür zusätzlich `pip install rasterio`):

```
lon, lat, z = lade_hoehenmodell("Copernicus_DSM_COG_10_N50_00_E012_00_DEM.tif",
```

```
                                (12.3, 50.2, 13.6, 50.8))   # Ausschnitt in Grad
schichten = hoehenschichten(lon, lat, z, [400, 500, 600, 700, 800, 900, 1000, 1100, 1200])
```

8.2 Autobahnen und Bahnstrecken

Die Verkehrskarte nutzt dieselbe Datendatei. Autobahnen werden einzeln gezeichnet und mit ihrer Nummer beschriftet, alle Bahnstrecken mit einem einzigen Befehl.

```
% Präambel: \input{deutschland_daten.tex}
\colorlet{autobahn}{akzentblau!85!black}
\begin{tikzpicture}[
    schild/.style = {fill=autobahn, text=white, font=\sffamily\tiny\bfseries,
                     rounded corners=1pt, inner sep=1.2pt},
  ]
  \GeoKoordinaten
  \clip \Rahmen;
  \foreach \staat in {DNK,POL,CZE,AUT,CHE,LIE,FRA,LUX,BEL,NLD}
    \fill[black!5, draw=white, line width=0.8pt] \Gebiet{\staat};
  \fill[black!10] \Gebiet{DEU};
  \foreach \land in {SH,HH,NI,HB,MV,BB,BE,ST,SN,TH,HE,NW,RP,SL,BW,BY}
    \draw[white, line width=0.6pt] \Gebiet{DE-\land};
  \draw[black!50, line width=0.7pt] \Gebiet{DEU};

  % --- Bahnstrecken: dünn, grau-schwarz -------------------------------
  \draw[black!45, line width=0.25pt, line join=round] \Bahn{alle};

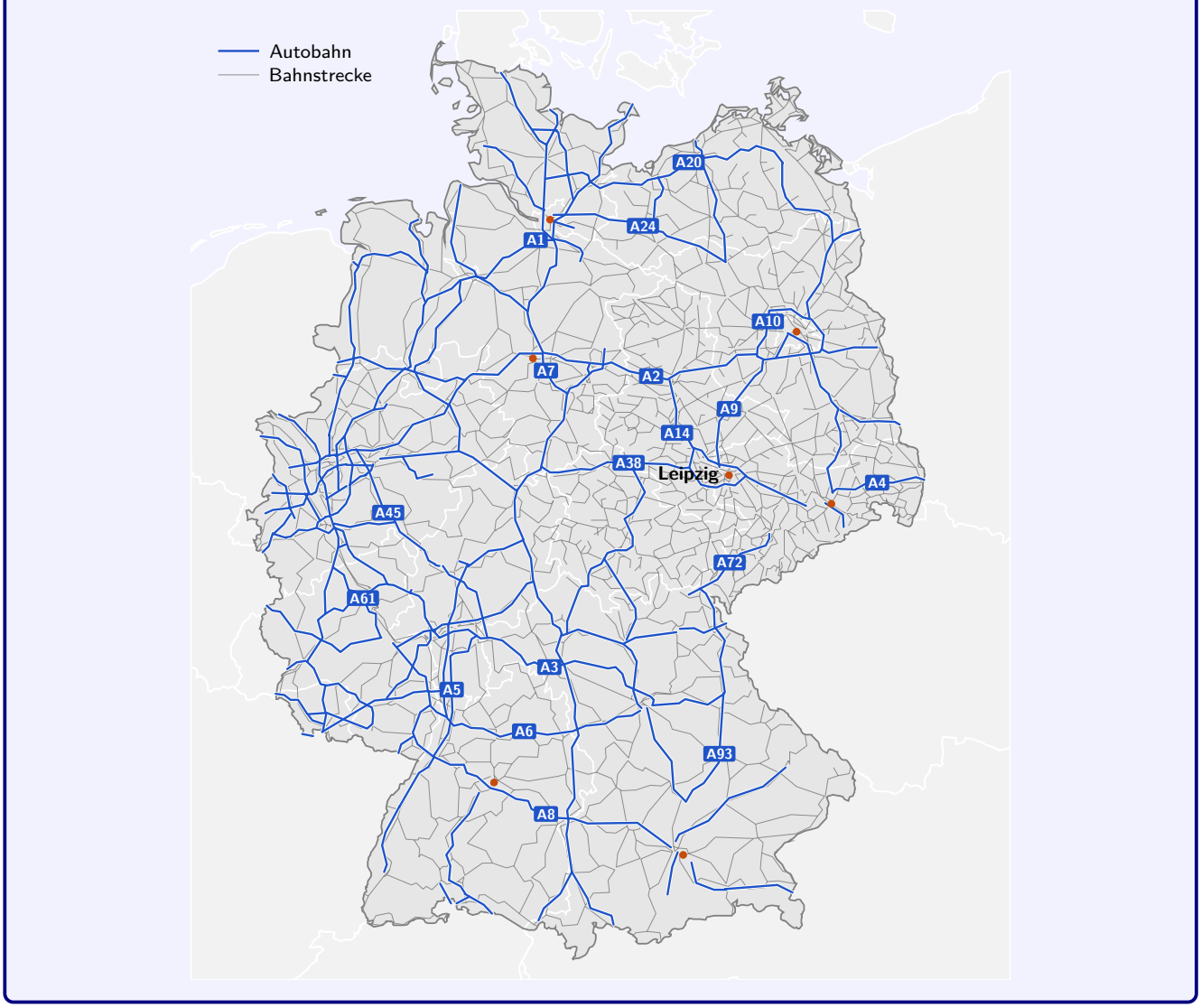
  % --- Autobahnen: alle aus der Liste, weißer Rand für bessere Lesbarkeit
  \foreach \a in \AutobahnListe
    \draw[white, line width=1.6pt, line join=round, line cap=round] \Autobahn{\a};
  \foreach \a in \AutobahnListe
    \draw[autobahn, line width=0.9pt, line join=round, line cap=round] \Autobahn{\a};

  % --- Nummernschilder an einigen wichtigen Autobahnen ------------------
  \foreach \a in {A1,A2,A3,A4,A5,A6,A7,A8,A9,A10,A14,A20,A24,A38,A45,A61,A72,A93}
    \node[schild] at (\a) {\a};

  % --- Großstädte -----------------------------------------------------
  \foreach \stadt/\lage in {Berlin/above left, Hamburg/above left, Muenchen/below right,
                            Leipzig/left, Dresden/right, Stuttgart/left, Hannover/left}
    \fill[akzentorange!85!black] (\stadt) circle (1.6pt);
  \node[left, font=\sffamily\scriptsize\bfseries] at (Leipzig) {Leipzig};

  % --- Legende --------------------------------------------------------
  \begin{scope}[shift={(0.4,13.6)}, font=\sffamily\scriptsize]
    \draw[autobahn, line width=0.9pt] (0,0) -- (0.6,0) node[right, black] {Autobahn};
    \draw[black!45, line width=0.25pt] (0,-0.35) -- (0.6,-0.35) node[right, black] {Bahnstrecke};
  \end{scope}
\end{tikzpicture}
```

Ausgabe



9 Europa und die ganze Welt

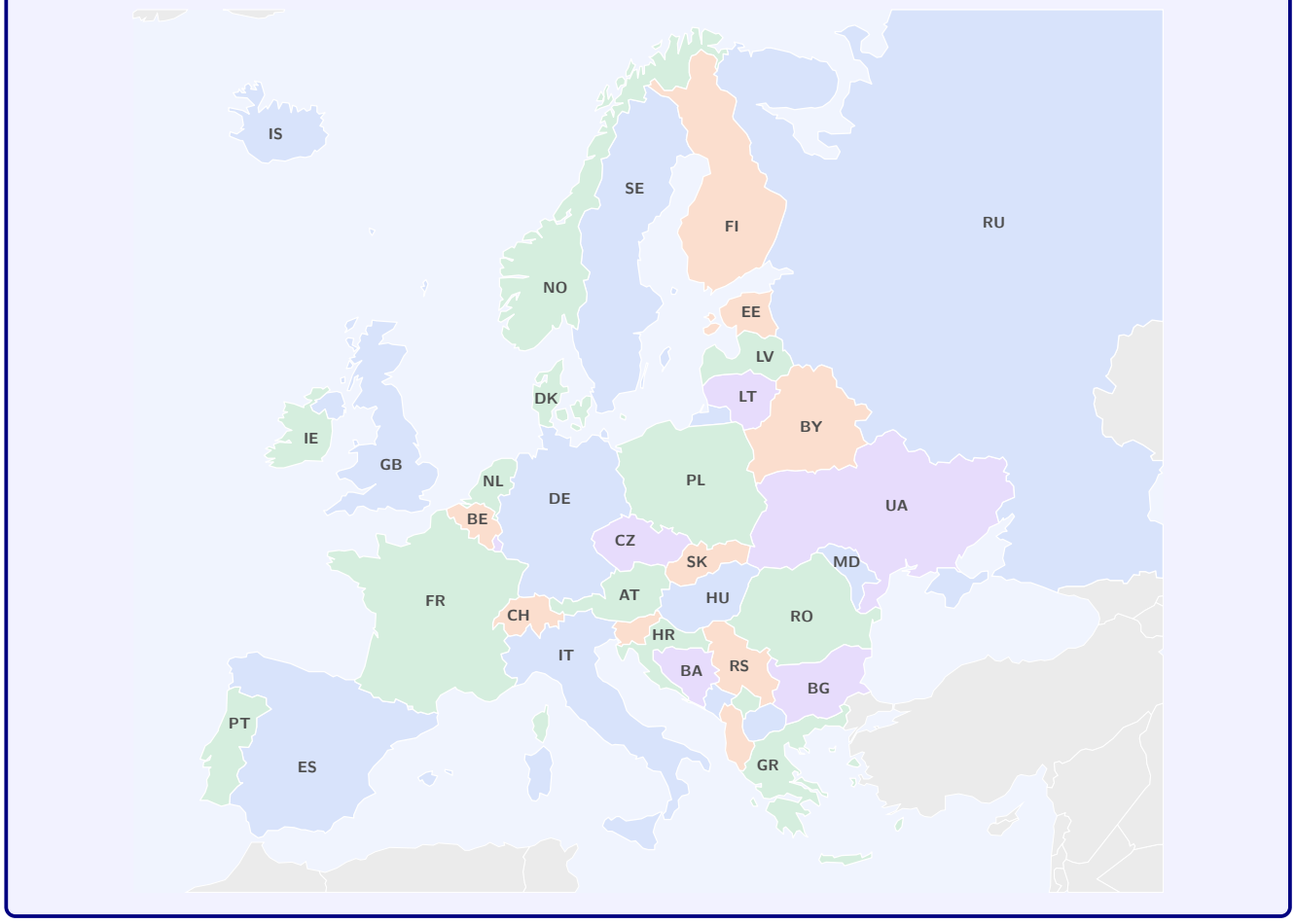
Die Natural-Earth-Datei `ne_50m_admin_0_countries` enthält alle Staaten der Erde mit einer Spalte `CONTINENT`. Ein Kontinent ist also nur ein Filter, im Skript eine einzige Zeile.

9.1 Europa: Färben ist ein Graphenproblem

Damit benachbarte Länder verschiedene Farben bekommen, fasst das Skript die Karte als **Graph** auf: Jedes Land ist ein Knoten, jede gemeinsame Grenze eine Kante. Gefärbt wird mit dem Verfahren **DSatur**: Als Nächstes kommt immer das Land an die Reihe, dessen Nachbarn schon die meisten verschiedenen Farben haben; es bekommt die kleinste noch freie Farbe. Für Europa reichen so **vier Farben**, wie es der Vierfarbensatz für jede Landkarte garantiert. Die Datendatei liefert das Ergebnis als Liste `\EuropaFarben`.

```
% Präambel: \input{europa_daten.tex}
\colorlet{farbe0}{akzentblau!18}
\colorlet{farbe1}{akzentgruen!18}
\colorlet{farbe2}{akzentorange!20}
\colorlet{farbe3}{akzentlila!18}
\colorlet{farbe4}{yellow!35}
\colorlet{meer}{akzentblau!7}
\begin{tikzpicture}
  \GeoKoordinaten
  \clip \Rahmen;
  \fill[meer] \Rahmen;
  % Länder außerhalb Europas grau
  \foreach \land in \AndereLaender
    \fill[black!8, draw=white, line width=0.4pt] \Gebiet{\land};
  % europäische Länder: Farbnummer kommt aus der Graphfärbung im Skript
  \foreach \land/\nr in \EuropaFarben
    \fill[farbe\nr, draw=white, line width=0.5pt, line join=round] \Gebiet{\land};
  % Länderkürzel (ISO 3166)
  \foreach \land/\iso in \EuropaKuerzel
    \node[font=\sffamily\tiny\bfseries, text=black!70] at (\land) {\iso};
\end{tikzpicture}
```

Ausgabe



9.2 Weltkarten: Jede Projektion lügt, aber anders

Eine Kugel lässt sich nicht verzerrungsfrei in die Ebene abwickeln. Man muss sich entscheiden, was erhalten bleiben soll:

- **Mercator** ist *winkeltreu* und deshalb für Seekarten und Online-Karten beliebt: $x = R\lambda$, $y = R \ln \tan\left(\frac{\pi}{4} + \frac{\varphi}{2}\right)$. Wegen $y'(\varphi) = \frac{R}{\cos\varphi}$ wird jede Strecke um den Faktor $\frac{1}{\cos\varphi}$ gestreckt, jede Fläche also um $\frac{1}{\cos^2\varphi}$; bei 75° Breite ist das schon das 15-Fache.
- **Robinson** ist ein Kompromiss: weder winkel- noch flächentreu, aber „sieht richtig aus“.
- **Equal Earth** ist *flächentreu*: Gleich große Gebiete erscheinen gleich groß.

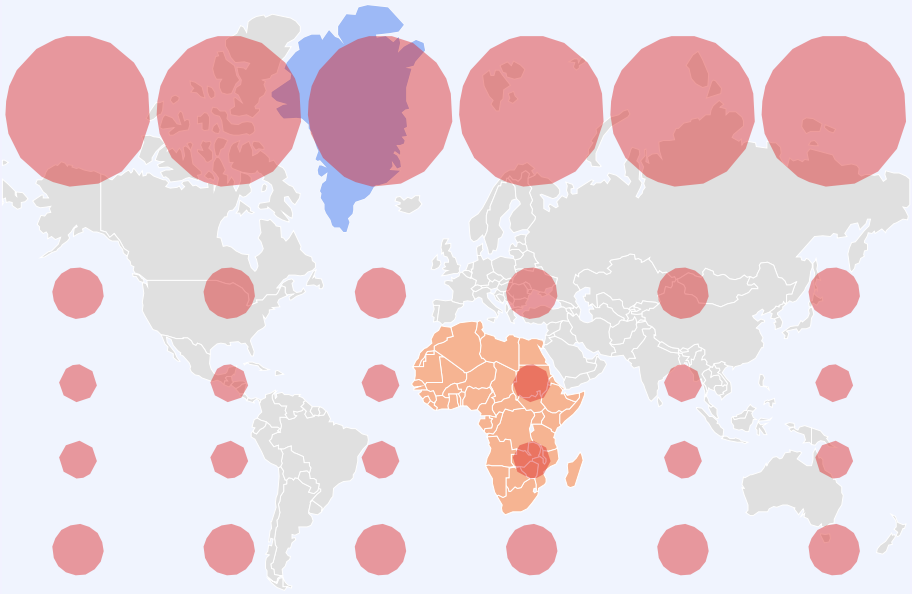
Die roten Flecken sind Kreise mit 800 km Radius auf der Erdkugel (**Tissot-Indikatrix**). Auf der Kugel sind sie alle gleich groß; auf der Karte zeigen sie, wie die Projektion verzerrt. Das Skript misst außerdem nach, wie viel größer Afrika (orange) auf der Karte ist als Grönland (blau). In Wirklichkeit ist Afrika rund 14-mal so groß.

	Mercator	**Robinson**	**Equal Earth**
Fläche Afrika : Fläche Grönland auf der Karte	0,9	7,2	13,6

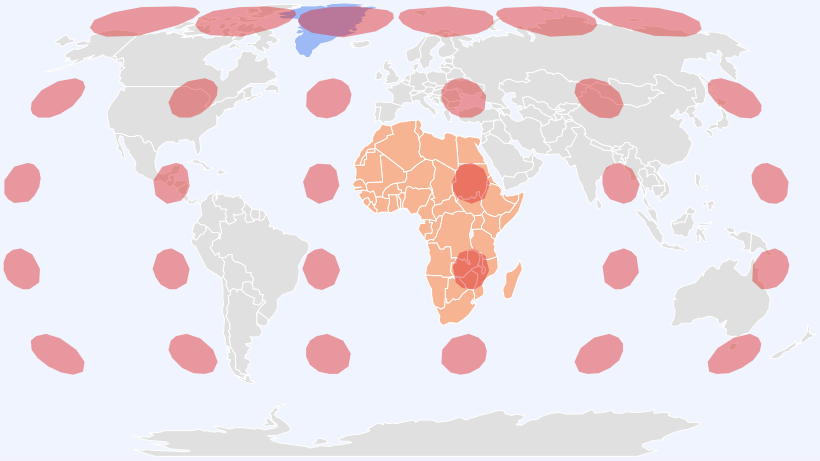
Die drei Datendateien definieren dieselben Befehle. Damit sie sich nicht überschreiben, wird jede in einer Gruppe geladen; das übernimmt der Befehl `\Weltkarte`:

```
\colorlet{meer}{akzentblau!7}
\newcommand\Weltkarte[2]{%
  \begingroup
  \input{welt_#1_daten.tex}%
  \begin{tikzpicture}
    \fill[meer] \Rahmen;
    \foreach \land in \GebietListe \fill[black!12, draw=white, line width=0.25pt] \Gebiet{\land};
    \foreach \land in \AfrikaListe \fill[akzentorange!45, draw=white, line width=0.25pt] \Gebiet
        {\land};
    \fill[akzentblau!45] \Gebiet{GRL};
    \foreach \k in \TissotListe \fill[akzentrot, fill opacity=0.45] \Tissot{\k};
    \node[anchor=north west, font=\sffamily\small\bfseries] at (0.1,-0.05) {#2};
  \end{tikzpicture}%
  \endgroup}
\Weltkarte{mercator}{Mercator}\par\medskip
\Weltkarte{robinson}{Robinson}\par\medskip
\Weltkarte{equalearth}{Equal Earth}
```

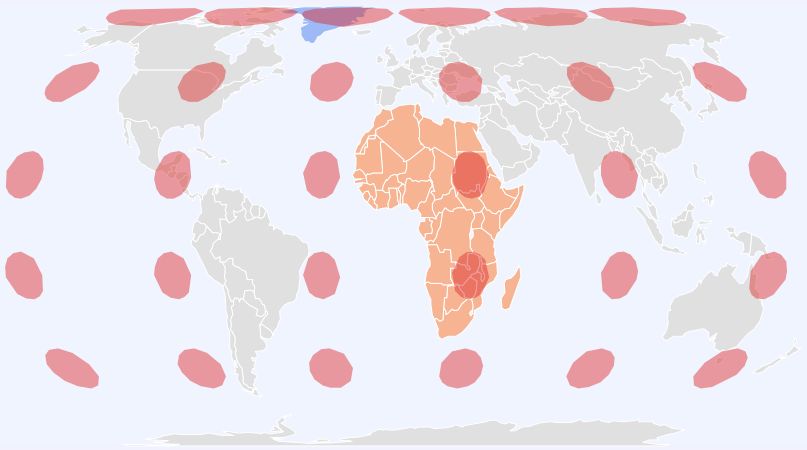
Ausgabe



Mercator



Robinson



Equal Earth

Die Vorlage `karte_welt_projektionen.tex` erzeugt dieselben drei Karten als drei Seiten eines `standalone`-Dokuments (Option `tikz`: pro `tikzpicture` eine Seite).

10 Historische Karten

Auch für historische Grenzen gibt es frei verfügbare Datensätze; sie sind aber *Rekonstruktionen*. Historische Grenzen waren oft unscharf, umstritten oder änderten sich mehrmals im Jahr. Jede Karte zeigt deshalb nur einen **Schnappschuss**, und die Daten sollten vor dem Unterrichtseinsatz mit einem Geschichtsatlas abgeglichen werden.

- **historical-basemaps** (A. Ourednik): Weltkarten zu über 50 Zeitpunkten von 123 000 v. Chr. bis 2010, u. a. für 1914, 1920, 1930, 1938, 1945 und 1960. Der Autor bezeichnet das Projekt ausdrücklich als „work in progress“.
- **CShapes 2.0** (ETH Zürich): Staatsgrenzen Jahr für Jahr von 1886 bis 2019, wissenschaftlich gepflegt (als R-Paket).
- **Ancient World Mapping Center** (AWMC): Antike, darunter das Römische Reich zu mehreren Zeitpunkten, Provinzen, Straßen, Aquädukte, antike Küstenlinien; Grundlage ist der *Barrington Atlas of the Greek and Roman World*.
- **Itiner-e**: das bisher vollständigste Netz der Römerstraßen (rund 300 000 km); **Pleiades**: antike Orte mit Koordinaten.

> **Lizenzen der historischen Daten:** Die Datendateien `historisch_daten.tex` (abgeleitet aus historical-basemaps) und `rom_daten.tex` (abgeleitet aus AWMC) stehen nicht unter CC BY wie der Rest dieses Repositorys, sondern unter der Lizenz ihrer Quelle: GPL-3.0 bzw. ODbL. Wer sie weitergibt oder verändert, muss die Quelle nennen und die Daten unter derselben Lizenz weitergeben. Auf den Karten selbst steht die Quellenangabe bereits.

10.1 Das Römische Reich um 200 n. Chr.

Reichsgebiet, Provinzgrenzen und Straßen stammen vom AWMC, die heutigen Küsten und Flüsse von Natural Earth. Die Städte sind mit ihren lateinischen Namen beschriftet (Colonia Agrippina = Köln, Augusta Treverorum = Trier, Augusta Vindelicum = Augsburg).

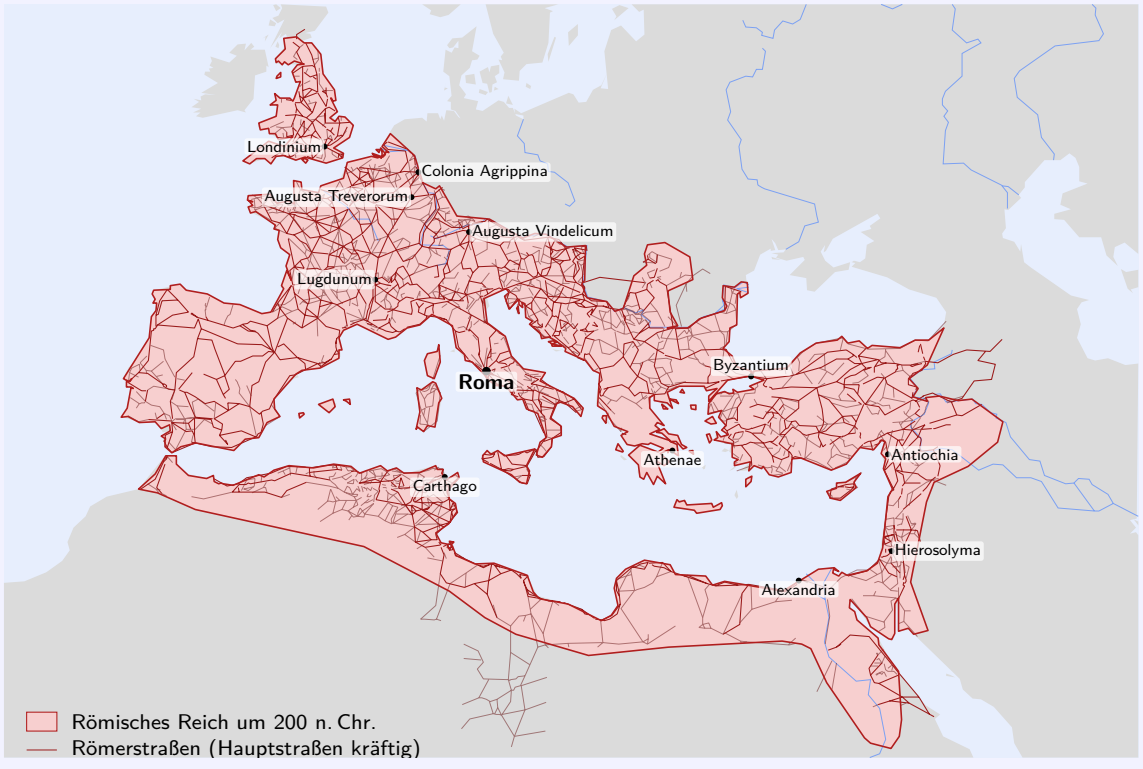
```
% Präambel: \input{rom_daten.tex}
\colorlet{meer}{akzentblau!11}
\colorlet{reich}{akzentrot!22}
\begin{tikzpicture}[
    stadt/.style = {font=\sffamily\tiny, inner sep=1pt, fill=white, fill opacity=0.7, text
        opacity=1, rounded corners=1pt},
  ]
  \GeoKoordinaten
  \clip \Rahmen;
  \fill[meer] \Rahmen;
  \fill[black!14] \Land{heute};                                    % heutige Küsten
  \fill[reich] \Reich{200};                                        % Reichsgebiet
```

```
  \draw[white, line width=0.4pt, line join=round] \Provinzgrenzen{200};
  \draw[akzentblau!60, line width=0.35pt] \Gewaesser{alle};          % große Flüsse
  \draw[akzentrot!45!black, line width=0.15pt, line join=round, opacity=0.5] \Strasse{neben};
  \draw[akzentrot!70!black, line width=0.4pt, line join=round] \Strasse{haupt};  % Hauptstraßen
  \draw[akzentrot!80!black, line width=0.6pt, line join=round] \Reich{200};

  % --- Städte (lateinische Namen) ------------------------------------
  \fill[black] (Roma) circle (1.6pt) node[below, stadt, font=\sffamily\scriptsize\bfseries] {Roma
      };
  \foreach \ort/\name/\lage in {Londinium/Londinium/left, ColoniaAgrippina/Colonia Agrippina/
      right,
        AugustaTreverorum/Augusta Treverorum/left, Lugdunum/Lugdunum/left,
        Carthago/Carthago/below, Alexandria/Alexandria/below, Athenae/Athenae/below,
        Antiochia/Antiochia/right, Byzantium/Byzantium/above, Hierosolyma/Hierosolyma/right,
        Augusta/Augusta Vindelicum/right}
    \fill (\ort) circle (1.1pt) node[\lage, stadt] {\name};

  % --- Legende -------------------------------------------------------
  \begin{scope}[shift={(0.3,0.35)}, font=\sffamily\scriptsize]
    \fill[reich, draw=akzentrot!80!black] (0,0) rectangle (0.4,0.25);
    \node[right] at (0.45,0.125) {Römisches Reich um 200 n.\,Chr.};
    \draw[akzentrot!70!black, line width=0.3pt] (0,-0.25) -- (0.4,-0.25);
    \node[right] at (0.45,-0.25) {Römerstraßen (Hauptstraßen kräftig)};
  \end{scope}
\end{tikzpicture}
```

Ausgabe



10.2 Deutschland 1938 – 1945 – 1949 bis 1990

Drei verkleinerte Teilkarten nebeneinander, jede in einem eigenen `scope`. Die gestrichelte Linie ist die heutige Grenze Deutschlands zum Vergleich.

```
% Präambel: \input{historisch_daten.tex}
\colorlet{meer}{akzentblau!7}
\begin{tikzpicture}[
    titel/.style = {font=\sffamily\bfseries\small, anchor=north west},
    klein/.style = {font=\sffamily\tiny, inner sep=1pt},
    heute/.style = {draw=black, line width=0.5pt, dash pattern=on 2pt off 1.2pt},
  ]

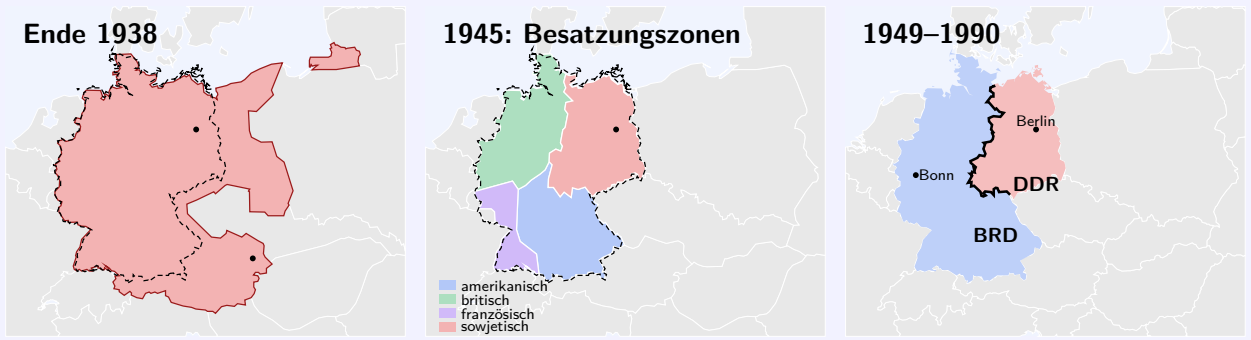
  % ================= 1) Ende 1938 ====================================
  \begin{scope}[scale=0.46]
    \GeoKoordinaten          % Punkte im verkleinerten Koordinatensystem anlegen
    \begin{scope}
      \clip \Rahmen;
      \fill[meer] \Rahmen;
      \foreach \g in \NachbarnReich \fill[black!9, draw=white, line width=0.4pt] \Gebiet{\g};
      \fill[akzentrot!35, draw=akzentrot!70!black, line width=0.5pt] \Gebiet{1938-Reich};
      \draw[heute] \Heute{DEU};
      \fill (Berlin) circle (2.5pt) (Wien) circle (2.5pt);
    \end{scope}
    \node[titel] at (0.2,9.7) {Ende 1938};
  \end{scope}

  % ================= 2) 1945: Besatzungszonen ========================
  \begin{scope}[xshift=5.8cm, scale=0.46]
    \GeoKoordinaten          % Punkte im verkleinerten Koordinatensystem anlegen
    \begin{scope}
      \clip \Rahmen;
      \fill[meer] \Rahmen;
      \foreach \g in \NachbarnZonen \fill[black!9, draw=white, line width=0.4pt] \Gebiet{\g};
      \foreach \z/\farbe in {USA/akzentblau, UK/akzentgruen, FRA/akzentlila, SOV/akzentrot}
        \fill[\farbe!35, draw=white, line width=0.5pt] \Gebiet{1945-\z};
      \draw[heute] \Heute{DEU};
      \fill (Berlin) circle (2.5pt);
    \end{scope}
    \node[titel] at (0.2,9.7) {1945: Besatzungszonen};
    \begin{scope}[shift={(0.4,0.5)}, font=\sffamily\tiny]
      \foreach \z/\farbe/\text [count=\i] in {USA/akzentblau/amerikanisch, UK/akzentgruen/
          britisch,
                                              FRA/akzentlila/französisch, SOV/akzentrot/sowjetisch
                                                 }
        \fill[\farbe!35] (0,1.3-0.42*\i) rectangle ++(0.5,0.3) node[right, black, inner sep=2pt]
            at (0.5,1.45-0.42*\i) {\text};
    \end{scope}
  \end{scope}

  % ================= 3) 1949-1990: BRD und DDR ========================
  \begin{scope}[xshift=11.6cm, scale=0.46]
    \GeoKoordinaten          % Punkte im verkleinerten Koordinatensystem anlegen
    \begin{scope}
      \clip \Rahmen;
      \fill[meer] \Rahmen;
      \foreach \g in \NachbarnHeute \fill[black!9, draw=white, line width=0.4pt] \Gebiet{\g};
      \fill[akzentblau!30] \Heute{West};
      \fill[akzentrot!30] \Heute{Ost};
      \draw[black, line width=1pt] \Heute{Grenze};
      \fill (Bonn) circle (2.5pt) (Berlin) circle (2.5pt);
    \end{scope}
    \node[titel] at (0.2,9.7) {1949--1990};
    \node[klein, right] at (Bonn) {Bonn};
    \node[klein, above] at (Berlin) {Berlin};
    \node[font=\sffamily\scriptsize\bfseries] at ($(Bonn)+(2.4,-1.8)$) {BRD};
    \node[font=\sffamily\scriptsize\bfseries] at ($(Leipzig)+(0.6,-0.6)$) {DDR};
  \end{scope}
```

```
  % gemeinsame Legende
  \draw[heute] (0,-0.35) -- (0.6,-0.35)
    node[right, font=\sffamily\tiny, black] {heutige Grenze Deutschlands zum Vergleich};
\end{tikzpicture}
```

Ausgabe



- **Ende 1938:** Der Datensatz für 1938 enthält bereits die Annexion Österreichs („Anschluss“, März 1938) und des Sudetengebiets (Oktober 1938), aber noch nicht das Memelland (März 1939). Das wurde durch Punkttests im Skript geprüft: Wien und Karlsbad liegen im Gebiet, Prag und Memel nicht.
- **1945:** die vier Besatzungszonen; die Gebiete östlich von Oder und Neiße gehören im Datensatz zu Polen bzw. zur Sowjetunion. Vereinfachungen: Berlin ist nicht in Sektoren geteilt, das Saarland ist bereits als eigenes Gebiet („Saar Protectorate“, eigentlich erst ab 1947) erfasst.
- **1949–1990:** Diese Karte braucht gar keinen historischen Datensatz. Die DDR besteht aus den heutigen Ländern MV, BB, BE, ST, SN und TH, die innerdeutsche Grenze ist die gemeinsame Grenze dieser Länder mit den alten Ländern. Zwei Ungenauigkeiten: West-Berlin wird mitgefärbt, und das Amt Neuhaus (östlich der Elbe) gehörte zur DDR, liegt aber seit 1993 in Niedersachsen.

Merke:

Historische Karten mit Ti*k*Z entstehen genauso wie aktuelle: GeoJSON laden, projizieren, vereinfachen, zeichnen. Der Unterschied liegt in der *Quellenkritik*: Wer hat die Grenze rekonstruiert, auf welcher Grundlage, und für welchen Stichtag?

11 Und wenn ich nur eine Grafik habe?

Aus einer Grafik lässt sich ebenfalls ein Polygonzug gewinnen; es gibt vier Wege. Für *echte* Grenzen sind die Geodaten aus Kapitel 2 aber fast immer besser.

(1) Vektorgrafik (SVG) umwandeln. Viele Karten auf Wikimedia Commons sind SVG-Dateien. Mit dem Programm `svg2tikz` (als Inkscape-Erweiterung oder per `pip install svg2tikz`) wird daraus Ti*k*Z-Code. Nachteil: Die Koordinaten haben keinen geografischen Bezug, und man muss die Lizenz der Grafik beachten (oft CC BY-SA).

(2) Rastergrafik nachzeichnen lassen. In Inkscape wandelt *Pfad → Bitmap nachzeichnen* ein Bild (PNG, JPG) automatisch in Pfade um, danach geht es weiter wie in (1). Das liefert Bézierkurven statt eines sauberen Polygonzugs und braucht meist Nacharbeit.

(3) Von Hand digitalisieren. Das Bild wird in Ti*k*Z hinterlegt, ein Gitter darübergelegt, und man liest die Eckpunkte ab, ideal mit Schülerinnen und Schülern auf Millimeterpapier. Das Grundgerüst (mit einer eigenen Bilddatei `vorlage.png`):

```
\begin{tikzpicture}
  \node[anchor=south west, inner sep=0, opacity=0.5]
       at (0,0) {\includegraphics[width=10cm]{vorlage.png}};
  \draw[help lines, black!30, step=0.5] (0,0) grid (10,8);   % Ablesegitter
  \foreach \x in {0,...,10} \node[below, font=\tiny] at (\x,0) {\x};
  \foreach \y in {0,...,8}  \node[left,  font=\tiny] at (0,\y) {\y};
  \draw[red, thick] (2.1,3.4) -- (2.8,4.0) -- (3.5,3.9) -- ... -- cycle;
\end{tikzpicture}
```

(4) Georeferenzieren, der saubere Weg. Eine Kartengrafik (z. B. ein gemeinfreier Atlas aus dem 19. Jahrhundert) wird in **QGIS** über *Layer → Georeferenzierer* geladen. Man setzt **Passpunkte**: Punkte, die man auf der alten Karte *und* in der Wirklichkeit eindeutig erkennt (Flussmündungen, Grenzecken, Kirchtürme, Kreuzungen im Gradnetz). QGIS berechnet daraus eine Transformation und legt die Grafik passgenau auf echte Koordinaten. Danach zeichnet man die gesuchten Grenzen in einem neuen Vektorlayer nach, exportiert ihn als GeoJSON und verarbeitet ihn mit `geo2tikz.py` wie alle anderen Daten. So lassen sich auch Grenzen digitalisieren, für die es keine Datenbank gibt.

Mathematisch steckt hinter dem Georeferenzieren eine **affine Abbildung** von den Bildkoordinaten (u, v) in die Kartenkoordinaten (x, y):

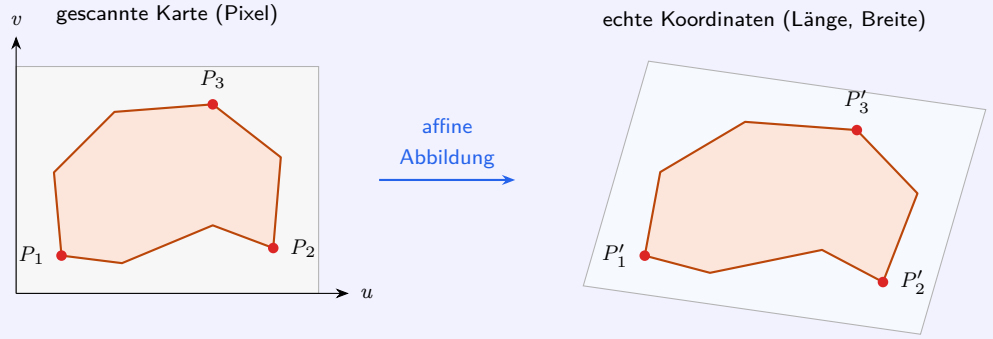
$$x = a\,u + b\,v + c, \qquad y = d\,u + e\,v + f.$$

Jeder Passpunkt liefert zwei Gleichungen. Mit **drei Passpunkten** entstehen zwei lineare Gleichungssysteme mit je drei Unbekannten (a, b, c bzw. d, e, f). Setzt man mehr Passpunkte, ist das System überbestimmt; QGIS wählt dann die Lösung mit den kleinsten Fehlerquadraten und zeigt für jeden Passpunkt den Restfehler an. So fallen falsch gesetzte Punkte sofort auf.

```
\begin{tikzpicture}[font=\sffamily\scriptsize, >={Stealth[length=5pt]},
                    pp/.style={circle, fill=akzentrot, inner sep=1.4pt}]
  \begin{scope}   % Bildkoordinaten
    \draw[black!30, fill=black!3] (0,0) rectangle (4,3);
    \draw[akzentorange!80!black, thick, fill=akzentorange!15, line join=round]
      (0.6,0.5) -- (1.4,0.4) -- (2.6,0.9) -- (3.4,0.6) -- (3.5,1.8) -- (2.6,2.5) -- (1.3,2.4) --
          (0.5,1.6) -- cycle;
```

```
    \node[pp, label=left:$P_1$] at (0.6,0.5) {};
    \node[pp, label=right:$P_2$] at (3.4,0.6) {};
    \node[pp, label=above:$P_3$] at (2.6,2.5) {};
    \draw[->] (0,0) -- (4.4,0) node[right] {$u$};
    \draw[->] (0,0) -- (0,3.4) node[above] {$v$};
    \node[above] at (2,3.4) {gescannte Karte (Pixel)};
  \end{scope}
  \draw[->, thick, akzentblau] (4.8,1.5) -- node[above, align=center] {affine\\Abbildung}
      (6.6,1.5);
  \begin{scope}[shift={(7.5,0.1)}, xslant=0.15, rotate=-8, xscale=1.15]   % echte Koordinaten
    \draw[black!30, fill=akzentblau!4] (0,0) rectangle (4,3);
    \draw[akzentorange!80!black, thick, fill=akzentorange!15, line join=round]
      (0.6,0.5) -- (1.4,0.4) -- (2.6,0.9) -- (3.4,0.6) -- (3.5,1.8) -- (2.6,2.5) -- (1.3,2.4) --
          (0.5,1.6) -- cycle;
    \node[pp, label=left:$P_1'$] at (0.6,0.5) {};
    \node[pp, label=right:$P_2'$] at (3.4,0.6) {};
    \node[pp, label=above:$P_3'$] at (2.6,2.5) {};
  \end{scope}
  \node at (9.9,3.6) {echte Koordinaten (Länge, Breite)};
\end{tikzpicture}
```

Ausgabe



Urheberrecht: Gedruckte und digitale Karten (Atlas, Schulbuch, Web-Karten) sind als Kartenwerke urheberrechtlich geschützt; auch abgezeichnete Umrisse können darunter fallen. Mit gemeinfreien Daten wie Natural Earth ist man auf der sicheren Seite. Alte Karten sind gemeinfrei, wenn der Urheber vor mehr als 70 Jahren gestorben ist; typische Vorlagen zum Georeferenzieren sind daher Atlanten des 19. und frühen 20. Jahrhunderts.

12 Ideen für den Unterricht

Mathematik: Maßstab und Flächeninhalt. Die Schülerinnen und Schüler legen die grobe Sachsen-Karte aus Kapitel 1 auf Karopapier, lesen die elf Eckpunkte ab und berechnen den Flächeninhalt mit der **Gaußschen Trapezformel**

$$A = \frac{1}{2}\left|\sum_{i=1}^{n}\left(x_i\,y_{i+1} - x_{i+1}\,y_i\right)\right|, \qquad (x_{n+1}, y_{n+1}) = (x_1, y_1).$$

Ergebnis: $A \approx 9{,}6\,\mathrm{cm}^2$. Mit dem Maßstab $1\,\mathrm{cm} \mathrel{\hat{=}} 43{,}1\,\mathrm{km}$ folgt $A \approx 9{,}6 \cdot 43{,}1^2\,\mathrm{km}^2 \approx 17\,900\,\mathrm{km}^2$, nur etwa 3 % weniger als der amtliche Wert. Anschließend lässt sich diskutieren, warum die Fläche mit mehr Punkten nicht *immer* größer wird (vgl. die Werte in Kapitel 4).

Mathematik: Analysis und Lineare Algebra. Die Mercator-Projektion ist eine Anwendung von ln, tan und der Kettenregel: Aus $y = R \ln \tan\left(\frac{\pi}{4} + \frac{\varphi}{2}\right)$ folgt $y'(\varphi) = \frac{R}{\cos\varphi}$ und damit die Flächenverzerrung $\frac{1}{\cos^2\varphi}$ (Kapitel 9). Höhenlinien sind Niveaulinien einer Funktion $f(x, y)$. Beim Georeferenzieren werden aus drei Passpunkten zwei lineare Gleichungssysteme, ein echter Anwendungsfall für LGS.

Informatik: Algorithmen. Der Douglas-Peucker-Algorithmus ist ein anschauliches Beispiel für *Rekursion* und *Teile und herrsche*; er lässt sich in Python in rund 15 Zeilen umsetzen. Die Einfärbung der Bundesländer führt auf das *Graphfärbungsproblem*: Länder sind Knoten, gemeinsame Grenzen sind Kanten. Reichen immer vier Farben? Das DSatur-Verfahren aus Kapitel 9 ist ein gieriger (greedy) Algorithmus und lässt sich gut nachspielen: Karte auf Papier, Buntstifte, Regel befolgen. Auch *Marching Squares* (Höhenlinien) eignet sich zum Nachvollziehen auf Karopapier, und GeoJSON ist ein gutes Beispiel für ein strukturiertes Datenformat.

Fächerübergreifend: Geschichte und Geografie. Die historischen Karten (Kapitel 10) laden zur Quellenkritik ein: Woher stammen die Grenzen, für welchen Stichtag gelten sie, was ist vereinfacht? Schülerinnen und Schüler können eine alte Karte selbst georeferenzieren und nachzeichnen und das Ergebnis mit dem Datensatz vergleichen. Stumme Karten (Kapitel 6.6) eignen sich für Topografie-Tests.

13 Datenquellen-Katalog

Alle hier genannten Quellen liefern Daten, die sich mit `geo2tikz.py` (bzw. nach einem Export als GeoJSON) in Ti*k*Z-Karten verwandeln lassen. Die Lizenz steht jeweils unter dem Namen; „Namensnennung“ heißt: Die Quelle muss unter der Karte genannt werden. Wo die Bedingungen je Datensatz unterschiedlich sind, steht „Bedingungen prüfen“.

Grenzen und Verwaltung	
Natural Earth gemeinfrei	Staaten, Bundesländer und Provinzen weltweit, Städte, Zeitzonen; drei Detailstufen. `https://www.naturalearthdata.com` GeoJSON: `https://github.com/nvkelso/natural-earth-vector`

BKG Geodatenzentrum DL-DE/BY-2.0	Amtliche Verwaltungsgrenzen Deutschlands bis zur Gemeinde (VG250, VG2500). `https://gdz.bkg.bund.de`
Eurostat GISCO Bedingungen prüfen	Statistische Regionen Europas (NUTS), passend zu Eurostat-Statistiken; ideal für eingefärbte Karten. `https://ec.europa.eu/eurostat/web/gisco`
Die Bundeswahlleiterin Bedingungen prüfen	Wahlkreise zur Bundestagswahl als Geodaten. `https://bundeswahlleiterin.de`
Open Data Leipzig Bedingungen prüfen	Kleinräumige Daten der Stadt Leipzig, z. B. Wahlbezirke, Einwohnerdichte. Viele Städte haben ähnliche Portale. `https://opendata.leipzig.de`
GADM nur nicht-kommerziell	Verwaltungsgrenzen aller Staaten bis zur Kreisebene. `https://gadm.org`

Gewässer, Relief, Natur

Natural Earth gemeinfrei	Flüsse, Seen, Küsten, Gebirge (Flächen), Gipfel, Meerestiefen-Stufen, Gletscher. `https://www.naturalearthdata.com`
Copernicus DEM frei, Namensnennung	Höhenmodell der ganzen Erde mit 30 m Auflösung, für Höhenlinien von Harz, Erzgebirge & Co. (Registrierung nötig). `https://dataspace.copernicus.eu`
ETOPO (NOAA) gemeinfrei	Weltweites Höhen- und Tiefenmodell (Land und Meer). `https://www.ncei.noaa.gov/products/etopo-global-relief-model`
fatiando-data frei	ETOPO1, auf 10 Bogenminuten gemittelt, als kleine Datei (2,7 MB); so in dieser Handreichung verwendet. `https://github.com/fatiando-data/earth-topography-10arcmin`
Tektonische Platten frei, Namensnennung	Plattengrenzen nach Bird (2003) als GeoJSON. `https://github.com/fraxen/tectonicplates`
USGS Erdbeben gemeinfrei	Aktuelle Erdbeben weltweit als GeoJSON-Feed (stündlich bis monatlich). `https://earthquake.usgs.gov/earthquakes/feed/`

Verkehr und Alltag

Natural Earth gemeinfrei	Hauptstraßen mit Nummern, Bahnstrecken, Flughäfen, Häfen. `https://www.naturalearthdata.com`

OpenStreetMap ODbL, Namensnennung	Alles bis ins Detail: jede Straße, Straßenbahnlinie, Schule, jeder Bach. `https://www.openstreetmap.org`
Geofabrik ODbL	Fertige OSM-Auszüge je Land und Bundesland zum Herunterladen. `https://download.geofabrik.de`
Overpass Turbo ODbL	Gezielte Abfrage einzelner OSM-Objekte (z. B. alle Straßenbahnlinien einer Stadt) mit Export als GeoJSON. `https://overpass-turbo.eu`

Geschichte

historical-basemaps GPL-3.0	Weltgrenzen zu über 50 Zeitpunkten (123 000 v. Chr. bis 2010), z. B. 200, 1500, 1914, 1938, 1945. `https://github.com/aourednik/historical-basemaps`
CShapes 2.0 Bedingungen prüfen	Staatsgrenzen Jahr für Jahr, 1886–2019 (ETH Zürich), als R-Paket. `https://cran.r-project.org/package=cshapes`
Ancient World Mapping Center ODbL, Namensnennung	Antike: Römisches Reich (60 v. Chr., 117 und 200 n. Chr., nach Diokletian), Provinzen, Straßen, Aquädukte, Reich Alexanders, Perserreich. `https://awmc.unc.edu/gis-data/` GeoJSON: `https://github.com/AWMC/geodata`
Itiner-e Bedingungen prüfen	Digitaler Atlas der Römerstraßen (ca. 300 000 km). `https://itiner-e.org`
Pleiades CC BY	Antike Orte mit Koordinaten. `https://pleiades.stoa.org`
David Rumsey Map Collection Lizenz je Karte	Zehntausende historische Karten als Vorlage zum Georeferenzieren. `https://www.davidrumsey.com`
Wikimedia Commons Lizenz je Datei	Historische und aktuelle Karten, viele als SVG. `https://commons.wikimedia.org`

Werkzeuge

QGIS frei (GPL)	Das Standardprogramm für Geodaten: anschauen, zuschneiden, georeferenzieren, nachzeichnen, als GeoJSON exportieren. `https://qgis.org`
mapshaper frei	Im Browser: Geodaten öffnen, vereinfachen, umwandeln (Shapefile ↔ GeoJSON ↔ SVG). `https://mapshaper.org`

geojson.io	GeoJSON im Browser ansehen und von Hand zeichnen.
frei	`https://geojson.io`
svg2tikz	SVG-Grafiken in Ti*k*Z-Code umwandeln.
frei (GPL)	`https://github.com/xyz2tex/svg2tikz`

14 Tipps und Stolpersteine

- **Alle Ebenen in einem Aufruf.** Bei `geo2tikz.py` haben getrennte Aufrufe verschiedene Maßstäbe; dann liegen Flüsse neben den Ländern.
- **Kompilierzeit:** Die Deutschland-Datendatei des Atlas enthält mit allen Ebenen rund 15 000 Punkte und kompiliert in wenigen Sekunden. Wird es zu langsam, die Toleranz erhöhen oder die Karte einmal als `standalone`-PDF erzeugen und mit `\includegraphics` einbinden.
- **Umlaute in Namen:** Namen von Gebieten, Flüssen und Punkten sind bewusst ohne Umlaute (`Muenchen`, `Mueritz`), weil Umlaute in `\csname` mit `pdflatex` Ärger machen.
- **Ränder:** Flüsse und Nachbarländer reichen über den Kartenrand hinaus; `\clip \Rahmen;` bzw. `\clip (0,0) rectangle (Breite,Höhe);` als erste Zeile schneidet sauber ab.
- **Füllen mit Löchern:** Hat ein Gebiet ein Loch (z. B. Brandenburg um Berlin), hilft `\fill[even odd rule]`. `geo2tikz.py` setzt die Option selbst.
- **Makronamen ohne Ziffern:** `\Gebiete1938` geht nicht, weil TeX-Befehlsnamen nur aus Buchstaben bestehen. Ziffern sind nur in Namen erlaubt, die über `\csname` laufen, also etwa `\Gebiet{1938-Reich}` oder `\Hoehe{500}`. Aus demselben Grund bestehen die Ebenennamen von `geo2tikz.py` nur aus Buchstaben.
- **Verkleinerte Teilkarten:** Bei `\begin{scope}[scale=0.46]` muss `\GeoKoordinaten` *innerhalb* des Scopes aufgerufen werden, sonst liegen die benannten Punkte an der unverkleinerten Stelle.
- **Mehrere Datendateien in einem Dokument:** jede in einer Gruppe (`\begingroup` … `\endgroup`) laden, siehe die Weltkarten in Kapitel 9. Bei `geo2tikz.py` verschiedene Ebenennamen wählen.
- **Mercator und Flächen über die Datumsgrenze:** Nach der Projektion können Flächen ungültig werden. `geo2tikz.py` repariert sie selbst; ältere Fassungen brachen hier mit „TopologyException: side location conflict“ ab.
- **Detailkarten** (z. B. Landkreise Sachsens) gibt Natural Earth nicht her; dafür die BKG-Daten (VG250) nehmen und mit `geo2tikz.py` umrechnen.

15 Zusammenfassung

- Karten nicht von Hand zeichnen, sondern freie Geodaten umrechnen: `geo2tikz.py` für eigene Ausschnitte, `karten_beispiele.py` für den fertigen Atlas.
- Drei Zutaten: Koordinaten, Projektion, Vereinfachung (Douglas-Peucker).
- `geo2tikz.py`: jede Ebene hat einen Befehl (`\geolaender`), einen Stil (`geo laender`) und Stile für jedes Objekt (`geo laender/DE-SN`); Punkte wie `(laender-DE-SN)` zum Beschriften.
- Atlas: `\Gebiet{...}`, `\Fluss{...}`, `\See{...}`, `\GeoKoordinaten`, dazu Höhen, Gebirge, Autobahnen und Bahn.
- Ebenen von unten nach oben zeichnen und am Kartenrand abschneiden.
- Projektion nach Zweck wählen und auf jede Karte eine Quellenzeile setzen; bei historischen Daten die Lizenz (GPL-3.0, ODbL) und die Quellenkritik beachten.

Kartendaten: Natural Earth (gemeinfrei), `https://www.naturalearthdata.com`; Höhen: ETOPO1, NOAA (gemeinfrei), über fatiando-data; Römisches Reich: © Ancient World Mapping Center (ODbL); Deutschland 1938/1945: historical-basemaps, A. Ourednik (GPL-3.0).